

for that group. The representatives will be the states of the reduced DFA M' . Let s be a representative state, and suppose on input a there is a transition of M from s to t . Let r be the representative of t 's group (r may be t). Then M' has a transition from s to r on a . Let the start state of M' be the representative of the group containing the start state s_0 of M , and let the accepting states of M' be the representatives that are in F . Note that each group of Π_{final} either consists only of states in F or has no states in F .

5. If M' has a dead state, that is, a state d that is not accepting and that has transitions to itself on all input symbols, then remove d from M' . Also remove any states not reachable from the start state. Any transitions to d from other states become undefined. $\square$

for each group G of Π do begin

partition G into subgroups such that two states s and t
of G are in the same subgroup if and only if for all
input symbols a , states s and t have transitions on a
to states in the same group of Π ;

/* at worst, a state will be in a subgroup by itself */
replace G in Π_{new} by the set of all subgroups formed

end

Fig. 3.45. Construction of Π_{new} .

Example 3.24. Let us reconsider the DFA represented in Fig. 3.29. The initial partition Π consists of two groups: (E) , the accepting state, and $(ABCD)$, the nonaccepting states. To construct Π_{new} , the algorithm of Fig. 3.45 first considers (E) . Since this group consists of a single state, it cannot be split further, so (E) is placed in Π_{new} . The algorithm then considers the group $(ABCD)$. On input a , each of these states has a transition to B , so they could all remain in one group as far as input a is concerned. On input b , however, A , B , and C go to members of the group $(ABCD)$ of Π , while D goes to E , a member of another group. Thus, in Π_{new} the group $(ABCD)$ must be split into two new groups (ABC) and (D) ; Π_{new} is thus $(ABC)(D)(E)$.

In the next pass through the algorithm of Fig. 3.45, we again have no splitting on input a , but (ABC) must be split into two new groups $(AC)(B)$, since on input b , A and C each have a transition to C , while B has a transition to D , a member of a group of the partition different from that of C . Thus the next value of Π is $(AC)(B)(D)(E)$.

In the next pass through the algorithm of Fig. 3.45, we cannot split any of the groups consisting of a single state. The only possibility is to try to split (AC) . But A and C go the same state B on input a , and they go to the same state C on input b . Hence, after this pass, $\Pi_{\text{new}} = \Pi$. Π_{final} is thus $(AC)(B)(D)(E)$.

If we choose *A* as the representative for the group (*AC*), and choose *B*, *D*, and *E* for the singleton groups, then we obtain the reduced automaton whose transition table is shown in Fig. 3.46. State *A* is the start state and state *E* is the only accepting state.

STATE	INPUT SYMBOL	
	<i>a</i>	<i>b</i>
<i>A</i>	<i>B</i>	<i>A</i>
<i>B</i>	<i>B</i>	<i>D</i>
<i>D</i>	<i>B</i>	<i>E</i>
<i>E</i>	<i>B</i>	<i>A</i>

Fig. 3.46. Transition table of reduced DFA.

For example, in the reduced automaton, state *E* has a transition to state *A* on input *b*, since *A* is the representative of the group for *C* and there is a transition from *E* to *C* on input *b* in the original automaton. A similar change has taken place in the entry for *A* and input *b*; otherwise, all other transitions are copied from Fig. 3.29. There is no dead state in Fig. 3.46, and all states are reachable from the start state *A*. □

State Minimization in Lexical Analyzers

To apply the state minimization procedure to the DFA's constructed in Section 3.7, we must begin Algorithm 3.5 with an initial partition that places in different groups all states indicating different tokens.

Example 3.25. In the case of the DFA of Fig. 3.37, the initial partition would group 0137 with 7, since they each gave no indication of a token recognized; 8 and 58 would also be grouped, since they each indicated token a^*b^+ . Other states would be in groups by themselves. Then we immediately discover that 0137 and 7 belong in different groups since they go to different groups on input *a*. Likewise, 8 and 58 do not belong together because of their transitions on input *b*. Thus the DFA of Fig. 3.37 is the minimum-state automaton doing its job. □

Table-Compression Methods

As we have indicated, there are many ways to implement the transition function of a finite automaton. The process of lexical analysis occupies a reasonable portion of the compiler's time, since it is the only process that must look at the input one character at a time. Therefore the lexical analyzer should minimize the number of operations it performs per input character. If a DFA is used to help implement the lexical analyzer, then an efficient representation of the transition function is desirable. A two-dimensional array, indexed by

states and characters, provides the fastest access, but it can take up too much space (say several hundred states by 128 characters). A more compact but slower scheme is to use a linked list to store the transitions out of each state, with a "default" transition at the end of the list. The most frequently occurring transition is one obvious choice for the default.

There is a more subtle implementation that combines the fast access of the array representation with the compactness of the list structures. Here we use a data structure consisting of four arrays indexed by state numbers as depicted in Fig. 3.47.⁷ The *base* array is used to determine the base location of the entries for each state stored in the *next* and *check* arrays. The *default* array is used to determine an alternative base location in case the current base location is invalid.

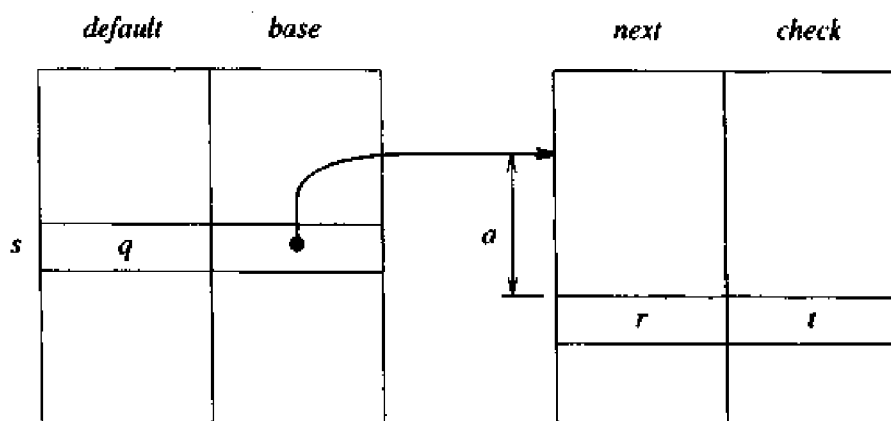


Fig. 3.47. Data structure for representing transition tables.

To compute $nextstate(s, a)$, the transition for state s on input symbol a , we first consult the pair of arrays *next* and *check*. In particular, we find their entries for state s in location $l = base[s] + a$, where character a is treated as an integer. We take $next[l]$ to be the next state for s on input a if $check[l] = s$. If $check[l] \neq s$, we determine $q = default[s]$ and repeat the entire procedure recursively, using q in place of s . The procedure is the following:

```

procedure nextstate( $s, a$ );
    if  $check[base[s] + a] = s$  then
        return  $next[base[s] + a]$ 
    else
        return nextstate( $default[s], a$ )

```

The intended use of the structure of Fig. 3.47 is to make the *next-check* arrays short, by taking advantage of the similarities among states. For

⁷ There would be in practice another array indexed by s , giving the pattern that matches, if any, when state s is entered. This information is derived from the NFA states making up DFA state s .

example, state q , the default for state s , might be the state that says we are "working on an identifier," such as state 10 in Fig. 3.13. Perhaps s is entered after seeing the prefix of the keyword then as well as a prefix of an identifier. On input character e we must go to a special state that remembers we have seen the, but otherwise state s behaves as state q does. Thus, we set $check[base[s]+e]$ to s and $next[base[s]+e]$ to the state for the.

While we may not be able to choose *base* values so that no *next-check* entries remain unused, experience shows that the simple strategy of setting the *base* to the lowest number such that the special entries can be filled in without conflicting with existing entries is fairly good and utilizes little more space than the minimum possible.

We can shorten *check* into an array indexed by states if the DFA has the property that the incoming edges to each state t all have the same label a . To implement this scheme, we set $check[t] = a$ and replace the test on line 2 of procedure *nextstate* by

if $check[next[base[s]+a]] = a$ then

EXERCISES

3.1 What is the input alphabet of each of the following languages?

- a) Pascal
- b) C
- c) Fortran 77
- d) Ada
- e) Lisp

3.2 What are the conventions regarding the use of blanks in each of the languages of Exercise 3.1?

3.3 Identify the lexemes that make up the tokens in the following programs. Give reasonable attribute values for the tokens.

a) Pascal

```
function max ( i, j : integer ) : integer ;
{ return maximum of integers i and j }
begin
    if i > j then max := i
    else max := j
end;
```

b) C

```
int max ( i, j ) int i, j;
/* return maximum of integers i and j */
{
    return i>j?i:j;
}
```

c) Fortran 77

```

      FUNCTION MAX ( I, J )
C     RETURN MAXIMUM OF INTEGERS I AND J
      IF ( I .GT. J ) THEN
        MAX = I
      ELSE
        MAX = J
      END IF
      RETURN

```

- 3.4 Write a program for the function `nextchar()` of Section 3.4 using the buffering scheme with sentinels described in Section 3.2.
- 3.5 In a string of length n , how many of the following are there?
- prefixes
 - suffixes
 - substrings
 - proper prefixes
 - subsequences
- *3.6 Describe the languages denoted by the following regular expressions:
- $0(0|1)^*0$
 - $((\epsilon|0)1^*)^*$
 - $(0|1)^*0(0|1)(0|1)$
 - $0^*10^*10^*10^*$
 - $(00|11)^*((01|10)(00|11)^*(01|10)(00|11)^*)^*$
- *3.7 Write regular definitions for the following languages.
- All strings of letters that contain the five vowels in order.
 - All strings of letters in which the letters are in ascending lexicographic order.
 - Comments consisting of a string surrounded by `/*` and `*/` without an intervening `*/` unless it appears inside the quotes `"` and `"`.
- *d) All strings of digits with no repeated digit.
- All strings of digits with at most one repeated digit.
 - All strings of 0's and 1's with an even number of 0's and an odd number of 1's.
 - The set of chess moves, such as $p-k4$ or $kbp \times qn$.
 - All strings of 0's and 1's that do not contain the substring 011.
 - All strings of 0's and 1's that do not contain the subsequence 011.
- 3.8 Specify the lexical form of numeric constants in the languages of Exercise 3.1.
- 3.9 Specify the lexical form of identifiers and keywords in the languages of Exercise 3.1.

3.10 The regular expression constructs permitted by Lex are listed in Fig. 3.48 in decreasing order of precedence. In this table, c stands for any single character, r for a regular expression, and s for a string.

EXPRESSION	MATCHES	EXAMPLE
c	any non-operator character c	a
$\backslash c$	character c literally	$\backslash *$
$"s"$	string s literally	$"**"$
$.$	any character but newline	$a.+b$
$^$	beginning of line	abc
$\$$	end of line	$abc\$$
$[s]$	any character in s	$[abc]$
$[^s]$	any character not in s	$[^abc]$
r^*	zero or more r 's	a^*
r^+	one or more r 's	a^+
$r?$	zero or one r	$a?$
$r\{m,n\}$	m to n occurrences of r	$a\{1,5\}$
r_1r_2	r_1 then r_2	ab
$r_1 \mid r_2$	r_1 or r_2	$a \mid b$
(r)	r	$(a \mid b)$
r_1/r_2	r_1 when followed by r_2	$abc/123$

Fig. 3.48. Lex regular expressions.

a) The special meaning of the operator symbols

$\backslash \ " \ . \ ^ \ \$ \ [\] \ * \ + \ ? \ \{ \} \ ! \ /$

must be turned off if the operator symbol is to be used as a matching character. This can be done by quoting the character, using one of two styles of quotation. The expression $"s"$ matches the string s literally, provided no $"$ appears in s . For example, $"**"$ matches the string $**$. We could also have matched this string with the expression $\backslash*\backslash*$. Note that an unquoted $*$ is an instance of the Kleene closure operator. Write a Lex regular expression that matches the string $"\backslash"$.

b) In Lex, a *complemented* character class is a character class in which the first symbol is $^$. A complemented character class matches any character not in the class. Thus, $[^a]$ matches any character that is not an a , $[^A-Za-z]$ matches any character that is not an upper or lower case letter, and so on. Show that for every regular definition with complemented character classes there is an equivalent regular expression without complemented character classes.

- c) The regular expression $r\{m,n\}$ matches from m to n occurrences of the pattern r . For example, $a\{1,5\}$ matches a string of one to five a's. Show that for every regular expression containing repetition operators there is an equivalent regular expression without repetition operators.
 - d) The operator $^$ matches the leftmost end of a line. This is the same operator that introduces a complemented character class, but the context in which $^$ appears will always determine a unique meaning for this operator. The operator $\$$ matches the rightmost end of a line. For example, $^[^aeiou]*\$$ matches any line that does not contain a lower case vowel. For every regular expression containing the $^$ and $\$$ operators is there an equivalent regular expression without these operators?
- 3.11 Write a Lex program that copies a file, replacing each nonnull sequence of white space by a single blank.
- 3.12 Write a Lex program that copies a Fortran program, replacing all instances of `DOUBLE PRECISION` by `REAL`.
- 3.13 Use your specification for keywords and identifiers for Fortran 77 from Exercise 3.9 to identify the tokens in the following statements:

```
IF(I) = TOKEN
IF(I) ASSIGN5TOKEN
IF(I) 10,20,30
IF(I) GOTO15
IF(I) THEN
```

Can you write your specification for keywords and identifiers in Lex?

- 3.14 In the UNIX system, the shell command `sh` uses the operators in Fig. 3.49 in filename expressions to describe sets of filenames. For example, the filename expression `*.o` matches all filenames ending in `.o`; `sort.?` matches all filenames that are of the form `sort.c` where `c` is any character. Character classes may be abbreviated as in `[a-z]`. Show how shell filename expressions can be represented by regular expressions.
- 3.15 Modify Algorithm 3.1 to find the longest prefix of the input that is accepted by the DFA.
- 3.16 Construct nondeterministic finite automata for the following regular expressions using Algorithm 3.3. Show the sequence of moves made by each in processing the input string *ababbab*.
- a) $(a|b)^*$
 - b) $(a^*|b^*)^*$

EXPRESSION	MATCHES	EXAMPLE
's'	string <i>s</i> literally	'\'
\c	character <i>c</i> literally	'\'
*	any string	*.o
?	any character	sort1.?
[s]	any character in <i>s</i>	sort.[cso]

Fig. 3.49. Filename expressions in the program *sh*.

- c) $((\epsilon|a)b^*)^*$
- d) $(a|b)^*abb(a|b)^*$

- 3.17 Convert the NFA's in Exercise 3.16 into DFA's using Algorithm 3.2. Show the sequence of moves made by each in processing the input string *ababbab*.
- 3.18 Construct DFA's for the regular expressions in Exercise 3.16 using Algorithm 3.5. Compare the size of the DFA's with those constructed in Exercise 3.17.
- 3.19 Construct a deterministic finite automaton from the transition diagrams for the tokens in Fig. 3.10.
- 3.20 Extend the table of Fig. 3.40 to include the regular expression operators *?* and *+*.
- 3.21 Minimize the states in the DFA's of Exercise 3.18 using Algorithm 3.6.
- 3.22 We can prove that two regular expressions are equivalent by showing that their minimum-state DFA's are the same, except for state names. Using this technique, show that the following regular expressions are all equivalent.
- a) $(a|b)^*$
 - b) $(a^*|b^*)^*$
 - c) $((\epsilon|a)b^*)^*$
- 3.23 Construct minimum-state DFA's for the following regular expressions.
- a) $(a|b)^*a(a|b)$
 - b) $(a|b)^*a(a|b)(a|b)$
 - c) $(a|b)^*a(a|b)(a|b)(a|b)$
- **d) Prove that any deterministic finite automaton for the regular expression $(a|b)^*a(a|b)(a|b) \cdots (a|b)$, where there are $n-1$ $(a|b)$'s at the end, must have at least 2^n states.

- 3.24** Construct the representation of Fig. 3.47 for the transition table of Exercise 3.19. Pick default states and try the following two methods of constructing the *next* array and compare the amounts of space used:
- Starting with the densest states (those with the largest number of entries differing from their default states) first, place the entries for the states in the *next* array.
 - Place the entries for the states in the *next* array in a random order.
- 3.25** A variant of the table compression scheme of Section 3.9 would be to avoid a recursive *nextstate* procedure by using a fixed default location for each state. Construct the representation of Fig. 3.47 for the transition table of Exercise 3.19 using this nonrecursive technique. Compare the space requirements with those of Exercise 3.24.
- 3.26** Let $b_1 b_2 \cdots b_m$ be a pattern string, called a *keyword*. A *trie* for a keyword is a transition diagram with $m+1$ states in which each state corresponds to a prefix of the keyword. For $1 \leq s \leq m$, there is a transition from state $s-1$ to state s on symbol b_s . The start and final states correspond to the empty string and the complete keyword, respectively. The trie for the keyword *ababaa* is:



We now define a *failure function* f on each state of the transition diagram, except the start state. Suppose states s and t represent prefixes u and v of the keyword. Then, we define $f(s) = t$ if and only if v is the longest proper suffix of u that is also the prefix of the keyword. The failure function f for the above trie is

s	1	2	3	4	5	6
$f(s)$	0	0	1	2	3	1

For example, states 3 and 1 represent prefixes *aba* and *a* of the keyword *ababaa*. $f(3) = 1$ because *a* is the longest proper suffix of *aba* that is prefix of the keyword.

- Construct the failure function for the keyword *abababuab*.
- Let the states in the trie be $0, 1, \dots, m$, with 0 the start state. Show that the algorithm in Fig. 3.50 correctly computes the failure function.
- Show that in the overall execution of the algorithm in Fig. 3.50 the assignment statement $t := f(t)$ in the inner loop is executed at most m times.
- Show that the algorithm runs in $O(m)$ time.

```

/* compute failure function  $f$  for  $b_1 \cdots b_m$  */
 $t := 0$ ;  $f(1) := 0$ ;
for  $s := 1$  to  $m-1$  do begin
    while  $t > 0$  and  $b_{s+1} \neq b_{t+1}$  do  $t := f(t)$ ;
    if  $b_{s+1} = b_{t+1}$  then begin  $t := t+1$ ;  $f(s+1) := t$  end;
    else  $f(s+1) := 0$ 
end

```

Fig. 3.50. Algorithm to compute failure function for Exercise 3.26.

3.27 Algorithm KMP in Fig. 3.51 uses the failure function f constructed as in Exercise 3.26 to determine whether keyword $b_1 \cdots b_m$ is a substring of a target string $a_1 \cdots a_n$. States in the trie for $b_1 \cdots b_m$ are numbered from 0 to m as in Exercise 3.26(b).

```

/* does  $a_1 \cdots a_n$  contain  $b_1 \cdots b_m$  as a substring */
 $s := 0$ ;
for  $i := 1$  to  $n$  do begin
    while  $s > 0$  and  $a_i \neq b_{s+1}$  do  $s := f(s)$ ;
    if  $a_i = b_{s+1}$  then  $s := s+1$ 
    if  $s = m$  then return "yes"
end;
return "no"

```

Fig. 3.51. Algorithm KMP.

- a) Apply Algorithm KMP to determine whether *ababaa* is a substring of *abababaab*.
 - *b) Prove that Algorithm KMP returns "yes" if and only if $b_1 \cdots b_m$ is a substring of $a_1 \cdots a_n$.
 - *c) Show that Algorithm KMP runs in $O(m+n)$ time.
 - *d) Given a keyword y , show that the failure function can be used to construct, in $O(|y|)$ time, a DFA with $|y| + 1$ states for the regular expression $\cdot y \cdot$, where $\cdot$ stands for any input character.
- **3.28** Define the *period* of a string s to be an integer p such that s can be expressed as $(uv)^k u$, for some $k \geq 0$, where $|uv| = p$ and v is not the empty string. For example, 2 and 4 are periods of the string *abababa*.
- a) Show that p is a period of a string s if and only if $st = us$ for some strings t and u of length p .
 - b) Show that if p and q are periods of a string s and if $p+q \leq |s| + \gcd(p, q)$, then $\gcd(p, q)$ is a period of s , where $\gcd(p, q)$ is the greatest common divisor of p and q .

- c) Let $sp(s_i)$ be the smallest period of the prefix of length i of a string s . Show that the failure function f has the property that $f(j) = j - sp(s_{j-1})$.

***3.29** Let the *shortest repeating prefix* of a string s be the shortest prefix u of s such that $s = u^k$, for some $k \geq 1$. For example, ab is the shortest repeating prefix of $abababab$ and aba is the shortest repeating prefix of aba . Construct an algorithm that finds the shortest repeating prefix of a string s in $O(|s|)$ time. *Hint.* Use the failure function of Exercise 3.26.

3.30 A *Fibonacci string* is defined as follows:

$$\begin{aligned} s_1 &= b \\ s_2 &= a \\ s_k &= s_{k-1}s_{k-2}, \text{ for } k > 2. \end{aligned}$$

For example, $s_3 = ab$, $s_4 = aba$, and $s_5 = abaab$.

- What is the length of s_n ?
- **b)** What is the smallest period of s_n ?
- Construct the failure function for s_6 .
- *d)** Using induction, show that the failure function for s_n can be expressed by $f(j) = j - |s_{k-1}|$, where k is such that $|s_k| \leq j+1 < |s_{k+1}|$ for $1 \leq j \leq |s_n|$.
- Apply Algorithm KMP to determine whether s_6 is a substring of the target string s_7 .
- Construct a DFA for the regular expression $.^*s_6.^*$.
- **g)** In Algorithm KMP, what is the maximum number of consecutive applications of the failure function executed in determining whether s_k is a substring of the target string s_{k+1} ?

3.31 We can extend the trie and failure function concepts of Exercise 3.26 from a single keyword to a set of keywords as follows. Each state in the trie corresponds to a prefix of one or more keywords. The start state corresponds to the empty string, and a state that corresponds to a complete keyword is a final state. Additional states may be made final during the computation of the failure function. The transition diagram for the set of keywords {he, she, his, hers} is shown in Fig. 3.52.

For the trie we define a *transition function* g that maps state-symbol pairs to states such that $g(s, b_{j+1}) = s'$ if state s corresponds to a prefix $b_1 \cdots b_j$ of some keyword and s' corresponds to a prefix $b_1 \cdots b_j b_{j+1}$. If s_0 is the start state, we define $g(s_0, a) = s_0$ for all input symbols a that are not the initial symbol of any keyword. We then set $g(s, a) = \text{fail}$ for any transition not defined. Note that there are no *fail* transitions for the start state.

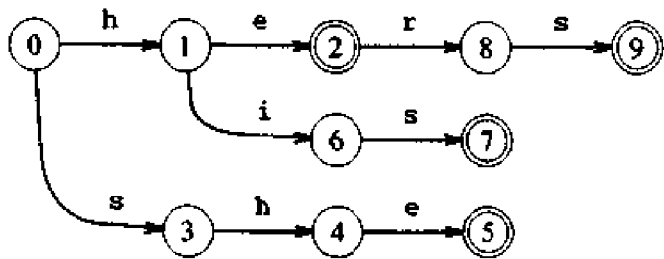


Fig. 3.52. Trie for keywords {he, she, his, hers}.

Suppose states s and t represent prefixes u and v of some keywords. Then, we define $f(s) = t$ if and only if v is the longest proper suffix of u that is also the prefix of some keyword. The failure function f for the transition diagram above is

s	1	2	3	4	5	6	7	8	9
$f(s)$	0	0	0	1	2	0	3	0	3

For example, states 4 and 1 represent prefixes sh and h . $f(4) = 1$ because h is the longest proper suffix of sh that is a prefix of some keyword. The failure function f can be computed for states of increasing depth using the algorithm in Fig. 3.53. The depth of a state is its distance from the start state.

```

for each state  $s$  of depth 1 do
     $f(s) := s_0$ ;
for each depth  $d \geq 1$  do
    for each state  $s_d$  of depth  $d$  and character  $a$ 
        such that  $g(s_d, a) = s'$  do begin
             $s := f(s_d)$ ;
            while  $g(s, a) = fail$  do  $s := f(s)$ ;
             $f(s') := g(s, a)$ ;
        end
    end
end
    
```

Fig. 3.53. Algorithm to compute failure function for trie of keywords.

Note that since $g(s_0, c) \neq fail$ for any character c , the while-loop in Fig. 3.53 is guaranteed to terminate. After setting $f(s')$ to $g(t, a)$, if $g(t, a)$ is a final state, we also make s' a final state, if it is not already.

- a) Construct the failure function for the set of keywords {aaa, abaaa, ababaaa}.

- *b) Show that the algorithm in Fig. 3.53 correctly computes the failure function.
- *c) Show that the failure function can be computed in time proportional to the sum of the lengths of the keywords.

3.32 Let g be the transition function and f the failure function of Exercise 3.31 for a set of keywords $K = \{y_1, y_2, \dots, y_k\}$. Algorithm AC in Fig. 3.54 uses g and f to determine whether a target string $a_1 \dots a_n$ contains a substring that is a keyword. State s_0 is the start state of the transition diagram for K , and F is the set of final states.

```

/* does  $a_1 \dots a_n$  contain a keyword as a substring */
 $s := s_0$ ;
for  $i := 1$  to  $n$  do begin
    while  $g(s, a_i) = \text{fail}$  do  $s = f(s)$ ;
     $s := g(s, a_i)$ ;
    if  $s$  is in  $F$  then return "yes"
end;
return "no"

```

Fig. 3.54. Algorithm AC.

- a) Apply Algorithm AC to the input string *ushers* using the transition and failure functions of Exercise 3.31.
 - *b) Prove that Algorithm AC returns "yes" if and only if some keyword y_i is a substring of $a_1 \dots a_n$.
 - *c) Show that Algorithm AC makes at most $2n$ state transitions in processing an input string of length n .
 - *d) Show that from the transition diagram and failure function for a set of keywords $\{y_1, y_2, \dots, y_k\}$ a DFA with at most $\sum_{i=1}^k |y_i| + 1$ states can be constructed in linear time for the regular expression $.(y_1|y_2|\dots|y_k).*$.
 - e) Modify Algorithm AC to print out each keyword found in the target string.
- 3.33** Use the algorithm in Exercise 3.32 to construct a lexical analyzer for the keywords in Pascal.
- 3.34** Define $lcs(x, y)$, a *longest common subsequence* of two strings x and y , to be a string that is a subsequence of both x and y and is as long as any such subsequence. For example, *tie* is a longest common subsequence of *striped* and *tiger*. Define $d(x, y)$, the *distance* between x and y , to be the minimum number of insertions and deletions required to transform x into y . For example, $d(\text{striped}, \text{tiger}) = 6$.

- a) Show that for any two strings x and y , the distance between x and y and the length of their longest common subsequence are related by $d(x, y) = |x| + |y| - (2 * |lcs(x, y)|)$.
- *b) Write an algorithm that takes two strings x and y as input and produces a longest common subsequence of x and y as output.
- 3.35** Define $e(x, y)$, the *edit distance* between two strings x and y , to be the minimum number of character insertions, deletions, and replacements that are required to transform x into y . Let $x = a_1 \cdots a_m$ and $y = b_1 \cdots b_n$. $e(x, y)$ can be computed by a dynamic programming algorithm using a distance array $d[0..m, 0..n]$ in which $d[i, j]$ is the edit distance between $a_1 \cdots a_i$ and $b_1 \cdots b_j$. The algorithm in Fig. 3.55 can be used to compute the d matrix. The function *repl* is just the cost of a character replacement: $repl(a_i, b_j) = 0$ if $a_i = b_j$, 1 otherwise.

```

for i := 0 to m do d[i, 0] := i;
for j := 1 to n do d[0, j] := j;
for i := 1 to m do
    for j := 1 to n do
        d[i, j] := min(d[i-1, j-1] + repl(a_i, b_j),
                       d[i-1, j] + 1,
                       d[i, j-1] + 1)

```

Fig. 3.55. Algorithm to compute edit distance between two strings.

- a) What is the relation between the distance metric of Exercise 3.34 and edit distance?
- b) Use the algorithm in Fig. 3.55 to compute the edit distance between *ababb* and *babaaa*.
- c) Construct an algorithm that prints out the minimal sequence of editing transformations required to transform x into y .
- 3.36** Give an algorithm that takes as input a string x and a regular expression r , and produces as output a string y in $L(r)$ such that $d(x, y)$ is as small as possible, where d is the distance function in Exercise 3.34.

PROGRAMMING EXERCISES

- P3.1** Write a lexical analyzer in Pascal or C for the tokens shown in Fig. 3.10.
- P3.2** Write a specification for the tokens of Pascal and from this specification construct transition diagrams. Use the transition diagrams to implement a lexical analyzer for Pascal in a language like C or Pascal.

- P3.3** Complete the Lex program in Fig. 3.18. Compare the size and speed of the resulting lexical analyzer produced by Lex with the program written in Exercise P3.1.
- P3.4** Write a Lex specification for the tokens of Pascal and use the Lex compiler to construct a lexical analyzer for Pascal.
- P3.5** Write a program that takes as input a regular expression and the name of a file, and produces as output all lines of the file that contain a substring denoted by the regular expression.
- P3.6** Add an error recovery scheme to the Lex program in Fig. 3.18 to enable it to continue to look for tokens in the presence of errors.
- P3.7** Program a lexical analyzer from the DFA constructed in Exercise 3.18 and compare this lexical analyzer with that constructed in Exercises P3.1 and P3.3.
- P3.8** Construct a tool that produces a lexical analyzer from a regular expression description of a set of tokens.

BIBLIOGRAPHIC NOTES

The restrictions imposed on the lexical aspects of a language are often determined by the environment in which the language was created. When Fortran was designed in 1954, punched cards were a common input medium. Blanks were ignored in Fortran partially because keypunchers, who prepared cards from handwritten notes, tended to miscount blanks (Backus [1981]). Algol 58's separation of the hardware representation from the reference language was a compromise reached after a member of the design committee insisted, "No! I will never use a period for a decimal point." (Wegstein [1981]).

Knuth [1973a] presents additional techniques for buffering input. Feldman [1979b] discusses the practical difficulties of token recognition in Fortran 77.

Regular expressions were first studied by Kleene [1956], who was interested in describing the events that could be represented by McCulloch and Pitts [1943] finite automaton model of nervous activity. The minimization of finite automata was first studied by Huffman [1954] and Moore [1956]. The equivalence of deterministic and nondeterministic automata as far as their ability to recognize languages was shown by Rabin and Scott [1959]. McNaughton and Yamada [1960] describe an algorithm to construct a DFA directly from a regular expression. More of the theory of regular expressions can be found in Hopcroft and Ullman [1979].

It was quickly appreciated that tools to build lexical analyzers from regular expression specifications would be useful in the implementation of compilers. Johnson et al. [1968] discuss an early such system. Lex, the language discussed in this chapter, is due to Lesk [1975], and has been used to construct lexical analyzers for many compilers using the UNIX system. The compact implementation scheme in Section 3.9 for transition tables is due to S. C.

Johnson, who first used it in the implementation of the Yacc parser generator (Johnson [1975]). Other table-compression schemes are discussed and evaluated in Dencker, Dürre, and Heuft [1984].

The problem of compact implementation of transition tables has been theoretically studied in a general setting by Tarjan and Yao [1979] and by Fredman, Komlós, and Szemerédi [1984]. Cormack, Horspool, and Kaiserswerth [1985] present a perfect hashing algorithm based on this work.

Regular expressions and finite automata have been used in many applications other than compiling. Many text editors use regular expressions for context searches. Thompson [1968], for example, describes the construction of an NFA from a regular expression (Algorithm 3.3) in the context of the QED text editor. The UNIX system has three general purpose regular expression searching programs: `grep`, `egrep`, and `fgrep`. `grep` does not allow union or parentheses for grouping in its regular expressions, but it does allow a limited form of backreferencing as in Snobol. `grep` employs Algorithms 3.3 and 3.4 to search for its regular expression patterns. The regular expressions in `egrep` are similar to those in `Lex`, except for iteration and lookahead. `egrep` uses a DFA with lazy state construction to look for its regular expression patterns, as outlined in Section 3.7. `fgrep` looks for patterns consisting of sets of keywords using the algorithm in Aho and Corasick [1975], which is discussed in Exercises 3.31 and 3.32. Aho [1980] discusses the relative performance of these programs.

Regular expressions have been widely used in text retrieval systems, in database query languages, and in file processing languages like AWK (Aho, Kernighan, and Weinberger [1979]). Jarvis [1976] used regular expressions to describe imperfections in printed circuits. Cherry [1982] used the keyword-matching algorithm in Exercise 3.32 to look for poor diction in manuscripts.

The string pattern matching algorithm in Exercises 3.26 and 3.27 is from Knuth, Morris, and Pratt [1977]. This paper also contains a good discussion of periods in strings. Another efficient algorithm for string matching was invented by Boyer and Moore [1977] who showed that a substring match can usually be determined without having to examine all characters of the target string. Hashing has also been proven as an effective technique for string pattern matching (Harrison [1971]).

The notion of a longest common subsequence discussed in Exercise 3.34 has been used in the design of the UNIX system file comparison program `diff` (Hunt and McIlroy [1976]). An efficient practical algorithm for computing longest common subsequences is described in Hunt and Szymanski [1977]. The algorithm for computing minimum edit distances in Exercise 3.35 is from Wagner and Fischer [1974]. Wagner [1974] contains a solution to Exercise 3.36. Sankoff and Kruskal [1983] contains a fascinating discussion of the broad range of applications of minimum distance recognition algorithms from the study of patterns in genetic sequences to problems in speech processing.

CHAPTER 4

Syntax Analysis

Every programming language has rules that prescribe the syntactic structure of well-formed programs. In Pascal, for example, a program is made out of blocks, a block out of statements, a statement out of expressions, an expression out of tokens, and so on. The syntax of programming language constructs can be described by context-free grammars or BNF (Backus-Naur Form) notation, introduced in Section 2.2. Grammars offer significant advantages to both language designers and compiler writers.

- A grammar gives a precise, yet easy-to-understand, syntactic specification of a programming language.
- From certain classes of grammars we can automatically construct an efficient parser that determines if a source program is syntactically well formed. As an additional benefit, the parser construction process can reveal syntactic ambiguities and other difficult-to-parse constructs that might otherwise go undetected in the initial design phase of a language and its compiler.
- A properly designed grammar imparts a structure to a programming language that is useful for the translation of source programs into correct object code and for the detection of errors. Tools are available for converting grammar-based descriptions of translations into working programs.
- Languages evolve over a period of time, acquiring new constructs and performing additional tasks. These new constructs can be added to a language more easily when there is an existing implementation based on a grammatical description of the language.

The bulk of this chapter is devoted to parsing methods that are typically used in compilers. We first present the basic concepts, then techniques that are suitable for hand implementation, and finally algorithms that have been used in automated tools. Since programs may contain syntactic errors, we extend the parsing methods so they recover from commonly occurring errors.

4.1 THE ROLE OF THE PARSER

In our compiler model, the parser obtains a string of tokens from the lexical analyzer, as shown in Fig. 4.1, and verifies that the string can be generated by the grammar for the source language. We expect the parser to report any syntax errors in an intelligible fashion. It should also recover from commonly occurring errors so that it can continue processing the remainder of its input.

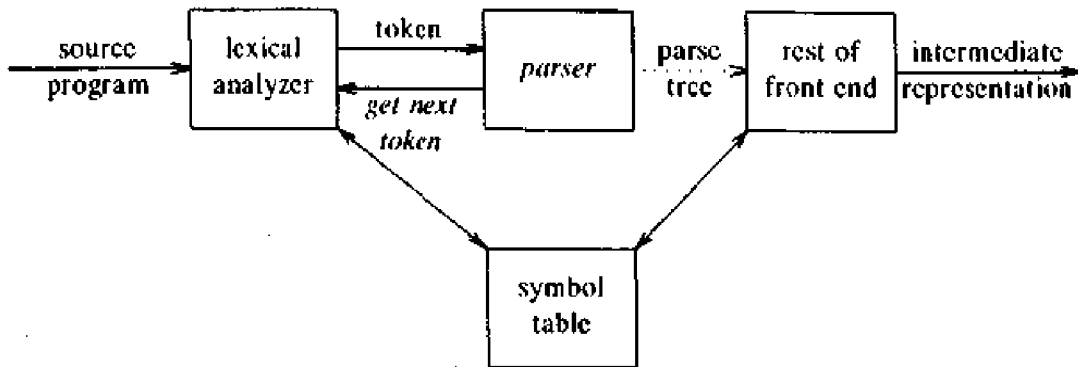


Fig. 4.1. Position of parser in compiler model.

There are three general types of parsers for grammars. Universal parsing methods such as the Cocke-Younger-Kasami algorithm and Earley's algorithm can parse any grammar (see the bibliographic notes). These methods, however, are too inefficient to use in production compilers. The methods commonly used in compilers are classified as being either top-down or bottom-up. As indicated by their names, top-down parsers build parse trees from the top (root) to the bottom (leaves), while bottom-up parsers start from the leaves and work up to the root. In both cases, the input to the parser is scanned from left to right, one symbol at a time.

The most efficient top-down and bottom-up methods work only on subclasses of grammars, but several of these subclasses, such as the LL and LR grammars, are expressive enough to describe most syntactic constructs in programming languages. Parsers implemented by hand often work with LL grammars; e.g., the approach of Section 2.4 constructs parsers for LL grammars. Parsers for the larger class of LR grammars are usually constructed by automated tools.

In this chapter, we assume the output of the parser is some representation of the parse tree for the stream of tokens produced by the lexical analyzer. In practice, there are a number of tasks that might be conducted during parsing, such as collecting information about various tokens into the symbol table, performing type checking and other kinds of semantic analysis, and generating intermediate code as in Chapter 2. We have lumped all of these activities into the "rest of front end" box in Fig. 4.1. We shall discuss these activities in detail in the next three chapters.

In the remainder of this section, we consider the nature of syntactic errors and general strategies for error recovery. Two of these strategies, called panic-mode and phrase-level recovery, are discussed in more detail together with the individual parsing methods. The implementation of each strategy calls upon the compiler writer's judgment, but we shall give some hints regarding approach.

Syntax Error Handling

If a compiler had to process only correct programs, its design and implementation would be greatly simplified. But programmers frequently write incorrect programs, and a good compiler should assist the programmer in identifying and locating errors. It is striking that although errors are so commonplace, few languages have been designed with error handling in mind. Our civilization would be radically different if spoken languages had the same requirement for syntactic accuracy as computer languages. Most programming language specifications do not describe how a compiler should respond to errors; the response is left to the compiler designer. Planning the error handling right from the start can both simplify the structure of a compiler and improve its response to errors.

We know that programs can contain errors at many different levels. For example, errors can be

- lexical, such as misspelling an identifier, keyword, or operator
- syntactic, such as an arithmetic expression with unbalanced parentheses
- semantic, such as an operator applied to an incompatible operand
- logical, such as an infinitely recursive call

Often much of the error detection and recovery in a compiler is centered around the syntax analysis phase. One reason for this is that many errors are syntactic in nature or are exposed when the stream of tokens coming from the lexical analyzer disobeys the grammatical rules defining the programming language. Another is the precision of modern parsing methods; they can detect the presence of syntactic errors in programs very efficiently. Accurately detecting the presence of semantic and logical errors at compile time is a much more difficult task. In this section, we present a few basic techniques for recovering from syntax errors; their implementation is discussed in conjunction with the parsing methods in this chapter.

The error handler in a parser has simple-to-state goals:

- It should report the presence of errors clearly and accurately.
- It should recover from each error quickly enough to be able to detect subsequent errors.
- It should not significantly slow down the processing of correct programs.

The effective realization of these goals presents difficult challenges.

Fortunately, common errors are simple ones and a relatively straightforward

error-handling mechanism often suffices. In some cases, however, an error may have occurred long before the position at which its presence is detected, and the precise nature of the error may be very difficult to deduce. In difficult cases, the error handler may have to guess what the programmer had in mind when the program was written.

Several parsing methods, such as the LL and LR methods, detect an error as soon as possible. More precisely, they have the *viable-prefix property*, meaning they detect that an error has occurred as soon as they see a prefix of the input that is not a prefix of any string in the language.

Example 4.1. To gain an appreciation of the kinds of errors that occur in practice, let us examine the errors Ripley and Druseikis [1978] found in a sample of student Pascal programs.

They discovered that errors do not occur that frequently; 60% of the programs compiled were syntactically and semantically correct. Even when errors did occur, they were quite sparse; 80% of the statements having errors had only one error, 13% had two. Finally, most errors were trivial; 90% were single token errors.

Many of the errors could be classified simply: 60% were punctuation errors, 20% operator and operand errors, 15% keyword errors, and the remaining five per cent other kinds. The bulk of the punctuation errors revolved around the incorrect use of semicolons.

For some concrete examples, consider the following Pascal program.

```
(1)    program prmax(input, output);
(2)    var
(3)        x, y: integer;

(4)    function max(i:integer; j:integer) : integer;
(5)    { return maximum of integers i and j }
(6)    begin
(7)        if i > j then max := i
(8)        else max := j
(9)    end;

(10)   begin
(11)       readln (x,y);
(12)       writeln (max(x,y))
(13)   end.
```

A common punctuation error is to use a comma in place of the semicolon in the argument list of a function declaration (e.g., using a comma in place of the first semicolon on line (4)); another is to leave out a mandatory semicolon at the end of a line (e.g., the semicolon at the end of line (4)); another is to put in an extraneous semicolon at the end of a line before an *else* (e.g., putting a semicolon at the end of line (7)).

Perhaps one reason why semicolon errors are so common is that the use of semicolons varies greatly from one language to another. In Pascal, a

semicolon is a statement separator; in PL/I and C, it is a statement terminator. Some studies have suggested that the latter usage is less error prone (Gannon and Horning [1975]).

A typical example of an operator error is to leave out the colon from `:=`. Misspellings of keywords are usually rare, but leaving out the `i` from `writeln` would be a representative example.

Many Pascal compilers have no difficulty handling common insertion, deletion, and mutation errors. In fact, several Pascal compilers will correctly compile the above program with a common punctuation or operator error; they will issue only a warning diagnostic, pinpointing the offending construct.

However, another common type of error is much more difficult to repair correctly. This is a missing `begin` or `end` (e.g., line (9) missing). Most compilers would not try to repair this kind of error. □

How should an error handler report the presence of an error? At the very least, it should report the place in the source program where an error is detected because there is a good chance that the actual error occurred within the previous few tokens. A common strategy employed by many compilers is to print the offending line with a pointer to the position at which an error is detected. If there is a reasonable likelihood of what the error actually is, an informative, understandable diagnostic message is also included; e.g., "semicolon missing at this position."

Once an error is detected, how should the parser recover? As we shall see, there are a number of general strategies, but no one method clearly dominates. In most cases, it is not adequate for the parser to quit after detecting the first error, because subsequent processing of the input may reveal additional errors. Usually, there is some form of error recovery in which the parser attempts to restore itself to a state where processing of the input can continue with a reasonable hope that correct input will be parsed and otherwise handled correctly by the compiler.

An inadequate job of recovery may introduce an annoying avalanche of "spurious" errors, those that were not made by the programmer, but were introduced by the changes made to the parser state during error recovery. In a similar vein, syntactic error recovery may introduce spurious semantic errors that will later be detected by the semantic analysis or code generation phases. For example, in recovering from an error, the parser may skip a declaration of some variable, say `zap`. When `zap` is later encountered in expressions, there is nothing syntactically wrong, but since there is no symbol-table entry for `zap`, a message "`zap` undefined" is generated.

A conservative strategy for a compiler is to inhibit error messages that stem from errors uncovered too close together in the input stream. After discovering one syntax error, the compiler should require several tokens to be parsed successfully before permitting another error message. In some cases, there may be too many errors for the compiler to continue sensible processing. (For example, how should a Pascal compiler respond to a Fortran program as

input?) It seems that an error-recovery strategy has to be a carefully considered compromise, taking into account the kinds of errors that are likely to occur and reasonable to process.

As we have mentioned, some compilers attempt error repair, a process in which the compiler attempts to guess what the programmer intended to write. The PL/C compiler (Conway and Wilcox [1973]) is an example of this type of compiler. Except possibly in an environment of short programs written by beginning students, extensive error repair is not likely to be cost effective. In fact, with the increasing emphasis on interactive computing and good programming environments, the trend seems to be toward simple error-recovery mechanisms.

Error-Recovery Strategies

There are many different general strategies that a parser can employ to recover from a syntactic error. Although no one strategy has proven itself to be universally acceptable, a few methods have broad applicability. Here we introduce the following strategies:

- panic mode
- phrase level
- error productions
- global correction

Panic-mode recovery. This is the simplest method to implement and can be used by most parsing methods. On discovering an error, the parser discards input symbols one at a time until one of a designated set of synchronizing tokens is found. The synchronizing tokens are usually delimiters, such as semicolon or **end**, whose role in the source program is clear. The compiler designer must select the synchronizing tokens appropriate for the source language, of course. While panic-mode correction often skips a considerable amount of input without checking it for additional errors, it has the advantage of simplicity and, unlike some other methods to be considered later, it is guaranteed not to go into an infinite loop. In situations where multiple errors in the same statement are rare, this method may be quite adequate.

Phrase-level recovery. On discovering an error, a parser may perform local correction on the remaining input; that is, it may replace a prefix of the remaining input by some string that allows the parser to continue. A typical local correction would be to replace a comma by a semicolon, delete an extraneous semicolon, or insert a missing semicolon. The choice of the local correction is left to the compiler designer. Of course, we must be careful to choose replacements that do not lead to infinite loops, as would be the case, for example, if we always inserted something on the input ahead of the current input symbol.

This type of replacement can correct any input string and has been used in several error-repairing compilers. The method was first used with top-down parsing. Its major drawback is the difficulty it has in coping with situations in

which the actual error has occurred before the point of detection.

Error productions. If we have a good idea of the common errors that might be encountered, we can augment the grammar for the language at hand with productions that generate the erroneous constructs. We then use the grammar augmented by these error productions to construct a parser. If an error production is used by the parser, we can generate appropriate error diagnostics to indicate the erroneous construct that has been recognized in the input.

Global correction. Ideally, we would like a compiler to make as few changes as possible in processing an incorrect input string. There are algorithms for choosing a minimal sequence of changes to obtain a globally least-cost correction. Given an incorrect input string x and grammar G , these algorithms will find a parse tree for a related string y , such that the number of insertions, deletions, and changes of tokens required to transform x into y is as small as possible. Unfortunately, these methods are in general too costly to implement in terms of time and space, so these techniques are currently only of theoretical interest.

We should point out that a closest correct program may not be what the programmer had in mind. Nevertheless, the notion of least-cost correction does provide a yardstick for evaluating error-recovery techniques, and it has been used for finding optimal replacement strings for phrase-level recovery.

4.2 CONTEXT-FREE GRAMMARS

Many programming language constructs have an inherently recursive structure that can be defined by context-free grammars. For example, we might have a conditional statement defined by a rule such as

$$\begin{aligned} &\text{If } S_1 \text{ and } S_2 \text{ are statements and } E \text{ is an expression, then} \\ &\quad \text{"if } E \text{ then } S_1 \text{ else } S_2\text{" is a statement.} \end{aligned} \quad (4.1)$$

This form of conditional statement cannot be specified using the notation of regular expressions; in Chapter 3, we saw that regular expressions can specify the lexical structure of tokens. On the other hand, using the syntactic variable *stmt* to denote the class of statements and *expr* the class of expressions, we can readily express (4.1) using the grammar production

$$stmt \rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \quad (4.2)$$

In this section, we review the definition of a context-free grammar and introduce terminology for talking about parsing. From Section 2.2, a context-free grammar (grammar for short) consists of terminals, nonterminals, a start symbol, and productions.

1. Terminals are the basic symbols from which strings are formed. The word "token" is a synonym for "terminal" when we are talking about grammars for programming languages. In (4.2), each of the keywords **if**, **then**, and **else** is a terminal.

2. Nonterminals are syntactic variables that denote sets of strings. In (4.2), *smt* and *expr* are nonterminals. The nonterminals define sets of strings that help define the language generated by the grammar. They also impose a hierarchical structure on the language that is useful for both syntax analysis and translation.
3. In a grammar, one nonterminal is distinguished as the start symbol, and the set of strings it denotes is the language defined by the grammar.
4. The productions of a grammar specify the manner in which the terminals and nonterminals can be combined to form strings. Each production consists of a nonterminal, followed by an arrow (sometimes the symbol $::=$ is used in place of the arrow), followed by a string of nonterminals and terminals.

Example 4.2. The grammar with the following productions defines simple arithmetic expressions.

```

expr  $\rightarrow$  expr op expr
expr  $\rightarrow$  ( expr )
expr  $\rightarrow$  - expr
expr  $\rightarrow$  id
  op  $\rightarrow$  +
  op  $\rightarrow$  -
  op  $\rightarrow$  *
  op  $\rightarrow$  /
  op  $\rightarrow$  †

```

In this grammar, the terminal symbols are

id + - * / † ()

The nonterminal symbols are *expr* and *op*, and *expr* is the start symbol. □

Notational Conventions

To avoid always having to state that “these are the terminals,” “these are the nonterminals,” and so on, we shall employ the following notational conventions with regard to grammars throughout the remainder of this book.

1. These symbols are terminals:

- i) Lower-case letters early in the alphabet such as *a*, *b*, *c*.
- ii) Operator symbols such as +, -, etc.
- iii) Punctuation symbols such as parentheses, comma, etc.
- iv) The digits 0, 1, . . . , 9.
- v) Boldface strings such as **id** or **if**.

2. These symbols are nonterminals:

- i) Upper-case letters early in the alphabet such as *A*, *B*, *C*.

- ii) The letter S , which, when it appears, is usually the start symbol.
 - iii) Lower-case italic names such as *expr* or *stmt*.
3. Upper-case letters late in the alphabet, such as X, Y, Z , represent *grammar symbols*, that is, either nonterminals or terminals.
 4. Lower-case letters late in the alphabet, chiefly $u, v, \dots, z$, represent strings of terminals.
 5. Lower-case Greek letters, α, β, γ , for example, represent strings of grammar symbols. Thus, a generic production could be written as $A \rightarrow \alpha$, indicating that there is a single nonterminal A on the left of the arrow (the *left side* of the production) and a string of grammar symbols α to the right of the arrow (the *right side* of the production).
 6. If $A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_k$ are all productions with A on the left (we call them *A-productions*), we may write $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_k$. We call $\alpha_1, \alpha_2, \dots, \alpha_k$ the *alternatives* for A .
 7. Unless otherwise stated, the left side of the first production is the start symbol.

Example 4.3. Using these shorthands, we could write the grammar of Example 4.2 concisely as

$$\begin{aligned} E &\rightarrow E A E \mid (E) \mid - E \mid \text{id} \\ A &\rightarrow + \mid - \mid * \mid / \mid \uparrow \end{aligned}$$

Our notational conventions tell us that E and A are nonterminals, with E the start symbol. The remaining symbols are terminals. $\square$

Derivations

There are several ways to view the process by which a grammar defines a language. In Section 2.2, we viewed this process as one of building parse trees, but there is also a related derivational view that we frequently find useful. In fact, this derivational view gives a precise description of the top-down construction of a parse tree. The central idea here is that a production is treated as a rewriting rule in which the nonterminal on the left is replaced by the string on the right side of the production.

For example, consider the following grammar for arithmetic expressions, with the nonterminal E representing an expression.

$$E \rightarrow E + E \mid E * E \mid (E) \mid - E \mid \text{id} \quad (4.3)$$

The production $E \rightarrow - E$ signifies that an expression preceded by a minus sign is also an expression. This production can be used to generate more complex expressions from simpler expressions by allowing us to replace any instance of an E by $- E$. In the simplest case, we can replace a single E by $- E$. We can describe this action by writing

$$E \Rightarrow -E$$

which is read “ E derives $-E$.” The production $E \rightarrow (E)$ tells us that we could also replace one instance of an E in any string of grammar symbols by (E) ; e.g., $E * E \Rightarrow (E) * E$ or $E * E \Rightarrow E * (E)$.

We can take a single E and repeatedly apply productions in any order to obtain a sequence of replacements. For example,

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(\text{id})$$

We call such a sequence of replacements a *derivation* of $-(\text{id})$ from E . This derivation provides a proof that one particular instance of an expression is the string $-(\text{id})$.

In a more abstract setting, we say that $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if $A \rightarrow \gamma$ is a production and α and β are arbitrary strings of grammar symbols. If $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$, we say α_1 *derives* α_n . The symbol $\Rightarrow$ means “derives in one step.” Often we wish to say “derives in zero or more steps.” For this purpose we can use the symbol $\stackrel{*}{\Rightarrow}$. Thus,

1. $\alpha \stackrel{*}{\Rightarrow} \alpha$ for any string α , and
2. If $\alpha \stackrel{*}{\Rightarrow} \beta$ and $\beta \Rightarrow \gamma$, then $\alpha \stackrel{*}{\Rightarrow} \gamma$.

Likewise, we use $\stackrel{+}{\Rightarrow}$ to mean “derives in one or more steps.”

Given a grammar G with start symbol S , we can use the $\stackrel{+}{\Rightarrow}$ relation to define $L(G)$, the *language generated by G* . Strings in $L(G)$ may contain only terminal symbols of G . We say a string of terminals w is in $L(G)$ if and only if $S \stackrel{+}{\Rightarrow} w$. The string w is called a *sentence* of G . A language that can be generated by a grammar is said to be a *context-free language*. If two grammars generate the same language, the grammars are said to be *equivalent*.

If $S \stackrel{*}{\Rightarrow} \alpha$, where α may contain nonterminals, then we say that α is a *sentential form* of G . A sentence is a sentential form with no nonterminals.

Example 4.4. The string $-(\text{id} + \text{id})$ is a sentence of grammar (4.3) because there is the derivation

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id}) \quad (4.4)$$

The strings E , $-E$, $-(E)$, $\dots$, $-(\text{id} + \text{id})$ appearing in this derivation are all sentential forms of this grammar. We write $E \stackrel{*}{\Rightarrow} -(\text{id} + \text{id})$ to indicate that $-(\text{id} + \text{id})$ can be derived from E .

We can show by induction on the length of a derivation that every sentence in the language of grammar (4.3) is an arithmetic expression involving the binary operators $+$ and $*$, the unary operator $-$, parentheses, and the operand id . Similarly, we can show by induction on the length of an arithmetic expression that all such expressions can be generated by this grammar. Thus, grammar (4.3) generates precisely the set of all arithmetic expressions involving binary $+$ and $*$, unary $-$, parentheses, and the operand id . $\square$

At each step in a derivation, there are two choices to be made. We need to choose which nonterminal to replace, and having made this choice, which

alternative to use for that nonterminal. For example, derivation (4.4) of Example 4.4 could continue from $-(E+E)$ as follows

$$-(E+E) \Rightarrow -(E+\text{id}) \Rightarrow -(\text{id}+\text{id}) \quad (4.5)$$

Each nonterminal in (4.5) is replaced by the same right side as in Example 4.4, but the order of replacements is different.

To understand how certain parsers work we need to consider derivations in which only the leftmost nonterminal in any sentential form is replaced at each step. Such derivations are termed *leftmost*. If $\alpha \Rightarrow \beta$ by a step in which the leftmost nonterminal in α is replaced, we write $\alpha \xRightarrow{\text{lm}} \beta$. Since derivation (4.4) is leftmost, we can rewrite it as:

$$E \xRightarrow{\text{lm}} -E \xRightarrow{\text{lm}} -(E) \xRightarrow{\text{lm}} -(E+E) \xRightarrow{\text{lm}} -(\text{id}+E) \xRightarrow{\text{lm}} -(\text{id}+\text{id})$$

Using our notational conventions, every leftmost step can be written $wA\gamma \xRightarrow{\text{lm}} w\delta\gamma$ where w consists of terminals only, $A \rightarrow \delta$ is the production applied, and γ is a string of grammar symbols. To emphasize the fact that α derives β by a leftmost derivation, we write $\alpha \xRightarrow{* \text{lm}} \beta$. If $S \xRightarrow{* \text{lm}} \alpha$, then we say α is a *left-sentential form* of the grammar at hand.

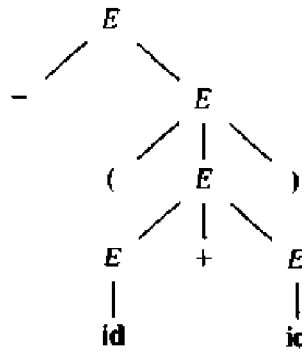
Analogous definitions hold for *rightmost* derivations in which the rightmost nonterminal is replaced at each step. Rightmost derivations are sometimes called *canonical* derivations.

Parse Trees and Derivations

A parse tree may be viewed as a graphical representation for a derivation that filters out the choice regarding replacement order. Recall from Section 2.2 that each interior node of a parse tree is labeled by some nonterminal A , and that the children of the node are labeled, from left to right, by the symbols in the right side of the production by which this A was replaced in the derivation. The leaves of the parse tree are labeled by nonterminals or terminals and, read from left to right, they constitute a sentential form, called the *yield* or *frontier* of the tree. For example, the parse tree for $-(\text{id}+\text{id})$ implied by derivation (4.4) is shown in Fig. 4.2.

To see the relationship between derivations and parse trees, consider any derivation $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$, where α_1 is a single nonterminal A . For each sentential form α_i in the derivation, we construct a parse tree whose yield is α_i . The process is an induction on i . For the basis, the tree for $\alpha_1 = A$ is a single node labeled A . To do the induction, suppose we have already constructed a parse tree whose yield is $\alpha_{i-1} = X_1X_2 \cdots X_k$. (Recalling our conventions, each X_i is either a nonterminal or a terminal.) Suppose α_i is derived from α_{i-1} by replacing X_j , a nonterminal, by $\beta = Y_1Y_2 \cdots Y_r$. That is, at the i th step of the derivation, production $X_j \rightarrow \beta$ is applied to α_{i-1} to derive $\alpha_i = X_1X_2 \cdots X_{j-1}\beta X_{j+1} \cdots X_k$.

To model this step of the derivation, we find the j th leaf from the left in the current parse tree. This leaf is labeled X_j . We give this leaf r children, labeled $Y_1, Y_2, \dots, Y_r$, from the left. As a special case, if $r = 0$, i.e.,

Fig. 4.2. Parse tree for $-(id + id)$.

$\beta = \epsilon$, then we give the j th leaf one child labeled ϵ .

Example 4.5. Consider derivation (4.4). The sequence of parse trees constructed from this derivation is shown in Fig. 4.3. In the first step of the derivation, $E \Rightarrow -E$. To model this step, we add two children, labeled $-$ and E , to the root E of the initial tree to create the second tree.

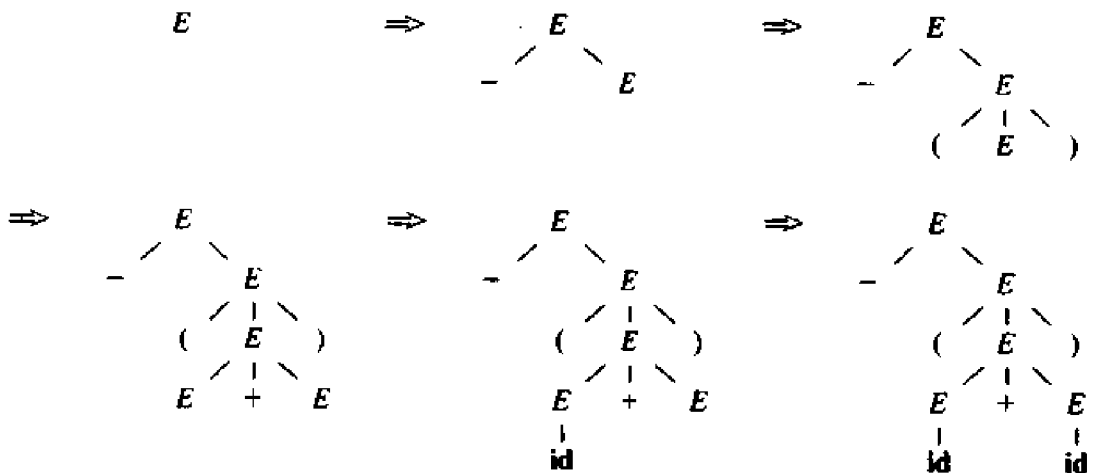


Fig. 4.3. Building the parse tree from derivation (4.4).

In the second step of the derivation, $-E \Rightarrow -(E)$. Consequently, we add three children, labeled $($, E , and $)$, to the leaf labeled E of the second tree to obtain the third tree with yield $-(E)$. Continuing in this fashion we obtain the complete parse tree as the sixth tree. $\square$

As we have mentioned, a parse tree ignores variations in the order in which symbols in sentential forms are replaced. For example, if derivation (4.4) were continued as in line (4.5), the same final parse tree of Fig. 4.3 would result. These variations in the order in which productions are applied can also be eliminated by considering only leftmost (or rightmost) derivations. It is

not hard to see that every parse tree has associated with it a unique leftmost and a unique rightmost derivation. In what follows, we shall frequently parse by producing a leftmost or rightmost derivation, understanding that instead of this derivation we could produce the parse tree itself. However, we should not assume that every sentence necessarily has only one parse tree or only one leftmost or rightmost derivation.

Example 4.6. Let us again consider the arithmetic expression grammar (4.3). The sentence $\text{id} + \text{id} * \text{id}$ has the two distinct leftmost derivations:

$$\begin{array}{ll}
 E \Rightarrow E + E & E \Rightarrow E * E \\
 \Rightarrow \text{id} + E & \Rightarrow E + E * E \\
 \Rightarrow \text{id} + E * E & \Rightarrow \text{id} + E * E \\
 \Rightarrow \text{id} + \text{id} * E & \Rightarrow \text{id} + \text{id} * E \\
 \Rightarrow \text{id} + \text{id} * \text{id} & \Rightarrow \text{id} + \text{id} * \text{id}
 \end{array}$$

with the two corresponding parse trees shown in Fig. 4.4. □

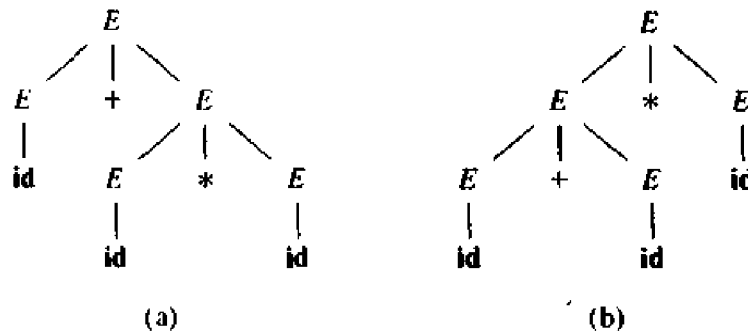


Fig. 4.4. Two parse trees for $\text{id} + \text{id} * \text{id}$.

Note that the parse tree of Fig. 4.4(a) reflects the commonly assumed precedence of $+$ and $*$, while the tree of Fig. 4.4(b) does not. That is, it is customary to treat operator $*$ as having higher precedence than $+$, corresponding to the fact that we would normally evaluate an expression like $a + b * c$ as $a + (b * c)$, rather than as $(a + b) * c$.

Ambiguity

A grammar that produces more than one parse tree for some sentence is said to be *ambiguous*. Put another way, an ambiguous grammar is one that produces more than one leftmost or more than one rightmost derivation for the same sentence. For certain types of parsers, it is desirable that the grammar be made unambiguous, for if it is not, we cannot uniquely determine which parse tree to select for a sentence. For some applications we shall also consider methods whereby we can use certain ambiguous grammars, together with *disambiguating rules* that “throw away” undesirable parse trees, leaving us with only one tree for each sentence.

4.3 WRITING A GRAMMAR

Grammars are capable of describing most, but not all, of the syntax of programming languages. A limited amount of syntax analysis is done by a lexical analyzer as it produces the sequence of tokens from the input characters. Certain constraints on the input, such as the requirement that identifiers be declared before they are used, cannot be described by a context-free grammar. Therefore, the sequences of tokens accepted by a parser form a superset of a programming language; subsequent phases must analyze the output of the parser to ensure compliance with rules that are not checked by the parser (see Chapter 6).

We begin this section by considering the division of work between a lexical analyzer and a parser. Because each parsing method can handle grammars only of a certain form, the initial grammar may have to be rewritten to make it parsable by the method chosen. Suitable grammars for expressions can often be constructed using associativity and precedence information, as in Section 2.2. In this section, we consider transformations that are useful for rewriting grammars so they become suitable for top-down parsing. We conclude this section by considering some programming language constructs that cannot be described by any grammar.

Regular Expressions vs. Context-Free Grammars

Every construct that can be described by a regular expression can also be described by a grammar. For example, the regular expression $(a|b)^*abb$ and the grammar

$$\begin{aligned} A_0 &\rightarrow aA_0 \mid bA_0 \mid aA_1 \\ A_1 &\rightarrow bA_2 \\ A_2 &\rightarrow bA_3 \\ A_3 &\rightarrow \epsilon \end{aligned}$$

describe the same language, the set of strings of a 's and b 's ending in abb .

We can mechanically convert a nondeterministic finite automaton (NFA) into a grammar that generates the same language as recognized by the NFA. The grammar above was constructed from the NFA of Fig. 3.23 using the following construction: For each state i of the NFA, create a nonterminal symbol A_i . If state i has a transition to state j on symbol a , introduce the production $A_i \rightarrow aA_j$. If state i goes to state j on input ϵ , introduce the production $A_i \rightarrow A_j$. If i is an accepting state, introduce $A_i \rightarrow \epsilon$. If i is the start state, make A_i be the start symbol of the grammar.

Since every regular set is a context-free language, we may reasonably ask, "Why use regular expressions to define the lexical syntax of a language?" There are several reasons.

1. The lexical rules of a language are frequently quite simple, and to describe them we do not need a notation as powerful as grammars.

2. Regular expressions generally provide a more concise and easier to understand notation for tokens than grammars.
3. More efficient lexical analyzers can be constructed automatically from regular expressions than from arbitrary grammars.
4. Separating the syntactic structure of a language into lexical and nonlexical parts provides a convenient way of modularizing the front end of a compiler into two manageable-sized components.

There are no firm guidelines as to what to put into the lexical rules, as opposed to the syntactic rules. Regular expressions are most useful for describing the structure of lexical constructs such as identifiers, constants, keywords, and so forth. Grammars, on the other hand, are most useful in describing nested structures such as balanced parentheses, matching begin-end's, corresponding if-then-else's, and so on. As we have noted, these nested structures cannot be described by regular expressions.

Verifying the Language Generated by a Grammar

Although compiler designers rarely do it for a complete programming language grammar, it is important to be able to reason that a given set of productions generates a particular language. Troublesome constructs can be studied by writing a concise, abstract grammar and studying the language that it generates. We shall construct such a grammar for conditionals below.

A proof that a grammar G generates a language L has two parts: we must show that every string generated by G is in L , and conversely that every string in L can indeed be generated by G .

Example 4.7. Consider the grammar (4.6)

$$S \rightarrow (S)S \mid \epsilon \quad (4.6)$$

It may not be initially apparent, but this simple grammar generates all strings of balanced parentheses, and only such strings. To see this, we shall show first that every sentence derivable from S is balanced, and then that every balanced string is derivable from S . To show that every sentence derivable from S is balanced, we use an inductive proof on the number of steps in a derivation. For the basis step, we note that the only string of terminals derivable from S in one step is the empty string, which surely is balanced.

Now assume that all derivations of fewer than n steps produce balanced sentences, and consider a leftmost derivation of exactly n steps. Such a derivation must be of the form

$$S \Rightarrow (S)S \xRightarrow{*} (x)S \xRightarrow{*} (x)y$$

The derivations of x and y from S take fewer than n steps so, by the inductive hypothesis, x and y are balanced. Therefore, the string $(x)y$ must be balanced.

We have thus shown that any string derivable from S is balanced. We must

next show that every balanced string is derivable from S . To do this we use induction on the length of a string. For the basis step, the empty string is derivable from S .

Now assume that every balanced string of length less than $2n$ is derivable from S , and consider a balanced string w of length $2n$, $n \geq 1$. Surely w begins with a left parenthesis. Let (x) be the shortest prefix of w having an equal number of left and right parentheses. Then w can be written as $(x)y$ where both x and y are balanced. Since x and y are of length less than $2n$, they are derivable from S by the inductive hypothesis. Thus, we can find a derivation of the form

$$S \Rightarrow (S)S \xRightarrow{*} (x)S \xRightarrow{*} (x)y$$

proving that $w = (x)y$ is also derivable from S . $\square$

Eliminating Ambiguity

Sometimes an ambiguous grammar can be rewritten to eliminate the ambiguity. As an example, we shall eliminate the ambiguity from the following "dangling-else" grammar:

$$\begin{array}{l} \text{stmt} \rightarrow \text{if expr then stmt} \\ \quad | \text{if expr then stmt else stmt} \\ \quad | \text{other} \end{array} \quad (4.7)$$

Here "other" stands for any other statement. According to this grammar, the compound conditional statement

$$\text{if } E_1 \text{ then } S_1 \text{ else if } E_2 \text{ then } S_2 \text{ else } S_3$$

has the parse tree shown in Fig. 4.5. Grammar (4.7) is ambiguous since the string

$$\text{if } E_1 \text{ then if } E_2 \text{ then } S_1 \text{ else } S_2 \quad (4.8)$$

has the two parse trees shown in Fig. 4.6.

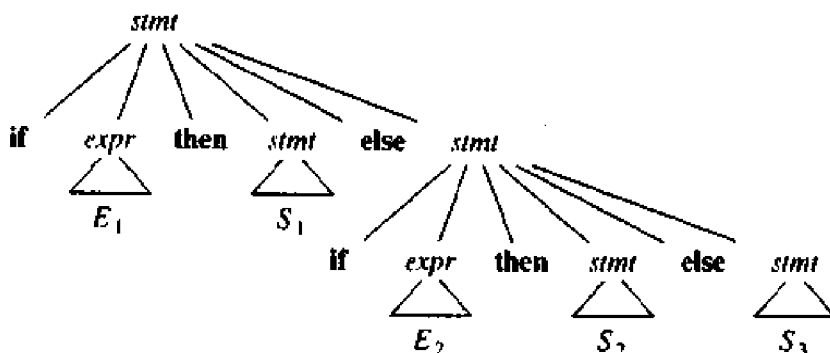


Fig. 4.5. Parse tree for conditional statement.

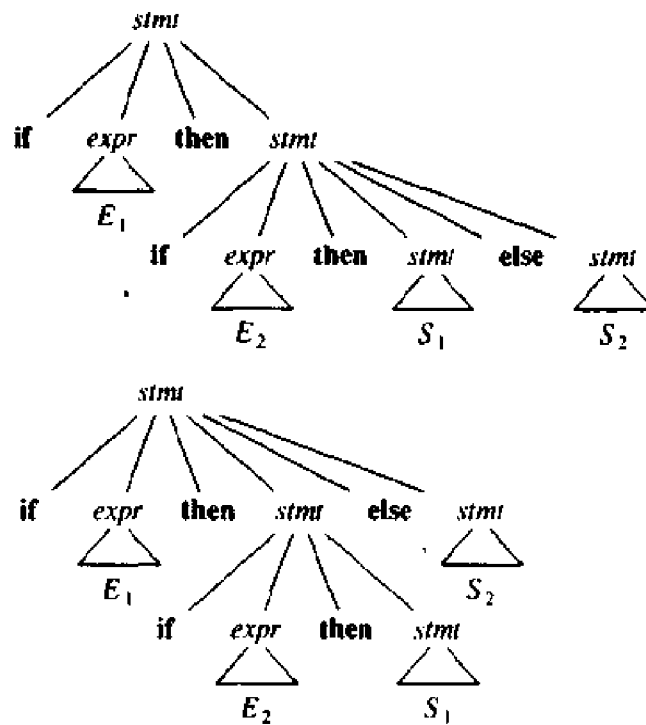


Fig. 4.6. Two parse trees for an ambiguous sentence.

In all programming languages with conditional statements of this form, the first parse tree is preferred. The general rule is, "Match each **else** with the closest previous unmatched **then**." This disambiguating rule can be incorporated directly into the grammar. For example, we can rewrite grammar (4.7) as the following unambiguous grammar. The idea is that a statement appearing between a **then** and an **else** must be "matched," i.e., it must not end with an unmatched **then** followed by any statement, for the **else** would then be forced to match this unmatched **then**. A matched statement is either an **if-then-else** statement containing no unmatched statements or it is any other kind of unconditional statement. Thus, we may use the grammar

$$\begin{aligned}
 \text{stmt} &\rightarrow \text{matched_stmt} \\
 &\quad | \text{unmatched_stmt} \\
 \text{matched_stmt} &\rightarrow \text{if expr then matched_stmt else matched_stmt} \\
 &\quad | \text{other} \\
 \text{unmatched_stmt} &\rightarrow \text{if expr then stmt} \\
 &\quad | \text{if expr then matched_stmt else unmatched_stmt}
 \end{aligned} \tag{4.9}$$

This grammar generates the same set of strings as (4.7), but it allows only one parsing for string (4.8), namely the one that associates each **else** with the closest previous unmatched **then**.

Elimination of Left Recursion

A grammar is *left recursive* if it has a nonterminal A such that there is a derivation $A \xRightarrow{+} A\alpha$ for some string α . Top-down parsing methods cannot handle left-recursive grammars, so a transformation that eliminates left recursion is needed. In Section 2.4, we discussed simple left recursion, where there was one production of the form $A \rightarrow A\alpha$. Here we study the general case. In Section 2.4, we showed how the left-recursive pair of productions $A \rightarrow A\alpha \mid \beta$ could be replaced by the non-left-recursive productions

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \epsilon \end{aligned}$$

without changing the set of strings derivable from A . This rule by itself suffices in many grammars.

Example 4.8. Consider the following grammar for arithmetic expressions.

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned} \quad (4.10)$$

Eliminating the *immediate left recursion* (productions of the form $A \rightarrow A\alpha$) to the productions for E and then for T , we obtain

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned} \quad (4.11)$$

□

No matter how many A -productions there are, we can eliminate immediate left recursion from them by the following technique. First, we group the A -productions as

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

where no β_i begins with an A . Then, we replace the A -productions by

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A' \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \epsilon \end{aligned}$$

The nonterminal A generates the same strings as before but is no longer left recursive. This procedure eliminates all immediate left recursion from the A and A' productions (provided no α_i is ϵ), but it does not eliminate left recursion involving derivations of two or more steps. For example, consider the grammar

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned} \quad (4.12)$$

The nonterminal S is left-recursive because $S \Rightarrow Aa \Rightarrow Sda$, but it is not

immediately left recursive.

Algorithm 4.1, below, will systematically eliminate left recursion from a grammar. It is guaranteed to work if the grammar has no cycles (derivations of the form $A \xRightarrow{+} A$) or ϵ -productions (productions of the form $A \rightarrow \epsilon$). Cycles can be systematically eliminated from a grammar as can ϵ -productions (see Exercises 4.20 and 4.22).

Algorithm 4.1. Eliminating left recursion.

Input. Grammar G with no cycles or ϵ -productions.

Output. An equivalent grammar with no left recursion.

Method. Apply the algorithm in Fig. 4.7 to G . Note that the resulting non-left-recursive grammar may have ϵ -productions. $\square$

1. Arrange the nonterminals in some order $A_1, A_2, \dots, A_n$.
2. **for** $i := 1$ **to** n **do begin**
 - for** $j := 1$ **to** $i - 1$ **do begin**
 - replace each production of the form $A_i \rightarrow A_j \gamma$
 - by the productions $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$,
 - where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ are all the current A_j -productions;
 - end**
 - eliminate the immediate left recursion among the A_i -productions
- end**

Fig. 4.7. Algorithm to eliminate left recursion from a grammar.

The reason the procedure in Fig. 4.7 works is that after the $i - 1^{\text{st}}$ iteration of the outer **for** loop in step (2), any production of the form $A_k \rightarrow A_l \alpha$, where $k < i$, must have $l > k$. As a result, on the next iteration, the inner loop (on j) progressively raises the lower limit on m in any production $A_i \rightarrow A_m \alpha$, until we must have $m \geq i$. Then, eliminating immediate left recursion for the A_i -productions forces m to be greater than i .

Example 4.9. Let us apply this procedure to grammar (4.12). Technically, Algorithm 4.1 is not guaranteed to work, because of the ϵ -production, but in this case the production $A \rightarrow \epsilon$ turns out to be harmless.

We order the nonterminals S, A . There is no immediate left recursion among the S -productions, so nothing happens during step (2) for the case $i = 1$. For $i = 2$, we substitute the S -productions in $A \rightarrow Sd$ to obtain the following A -productions.

$$A \rightarrow Ac \mid Aad \mid bd \mid \epsilon$$

Eliminating the immediate left recursion among the A -productions yields the following grammar.

$$\begin{aligned}
S &\rightarrow Aa \mid b \\
A &\rightarrow bdA' \mid A' \\
A' &\rightarrow cA' \mid adA' \mid \epsilon
\end{aligned}
\quad \square$$

Left Factoring

Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive parsing. The basic idea is that when it is not clear which of two alternative productions to use to expand a nonterminal A , we may be able to rewrite the A -productions to defer the decision until we have seen enough of the input to make the right choice.

For example, if we have the two productions

$$\begin{aligned}
stmt &\rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \\
&\quad \mid \text{if } expr \text{ then } stmt
\end{aligned}$$

on seeing the input token **if**, we cannot immediately tell which production to choose to expand *stmt*. In general, if $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ are two A -productions, and the input begins with a nonempty string derived from α , we do not know whether to expand A to $\alpha\beta_1$ or to $\alpha\beta_2$. However, we may defer the decision by expanding A to $\alpha A'$. Then, after seeing the input derived from α , we expand A' to β_1 or to β_2 . That is, left-factored, the original productions become

$$\begin{aligned}
A &\rightarrow \alpha A' \\
A' &\rightarrow \beta_1 \mid \beta_2
\end{aligned}$$

Algorithm 4.2. Left factoring a grammar.

Input. Grammar G .

Output. An equivalent left-factored grammar.

Method. For each nonterminal A find the longest prefix α common to two or more of its alternatives. If $\alpha \neq \epsilon$, i.e., there is a nontrivial common prefix, replace all the A productions $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \cdots \mid \alpha\beta_n \mid \gamma$ where γ represents all alternatives that do not begin with α by

$$\begin{aligned}
A &\rightarrow \alpha A' \mid \gamma \\
A' &\rightarrow \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n
\end{aligned}$$

Here A' is a new nonterminal. Repeatedly apply this transformation until no two alternatives for a nonterminal have a common prefix. $\square$

Example 4.10. The following grammar abstracts the dangling-else problem:

$$\begin{aligned}
S &\rightarrow iEtS \mid iEtSeS \mid a \\
E &\rightarrow b
\end{aligned}
\quad (4.13)$$

Here i , t , and e stand for **if**, **then** and **else**, E and S for “expression” and “statement.” Left-factored, this grammar becomes:

$$\begin{aligned}
 S &\rightarrow iEtSS' \mid a \\
 S' &\rightarrow eS \mid \epsilon \\
 E &\rightarrow b
 \end{aligned}
 \tag{4.14}$$

Thus, we may expand S to $iEtSS'$ on input i , and wait until $iEtS$ has been seen to decide whether to expand S' to eS or to ϵ . Of course, grammars (4.13) and (4.14) are both ambiguous, and on input e , it will not be clear which alternative for S' should be chosen. Example 4.19 discusses a way out of this dilemma. $\square$

Non-Context-Free Language Constructs

It should come as no surprise that some languages cannot be generated by any grammar. In fact, a few syntactic constructs found in many programming languages cannot be specified using grammars alone. In this section, we shall present several of these constructs, using simple abstract languages to illustrate the difficulties.

Example 4.11. Consider the abstract language $L_1 = \{wew \mid w \text{ is in } (a\{b\})^*\}$. L_1 consists of all words composed of a repeated string of a 's and b 's separated by a c , such as $aabcaab$. It can be proven this language is not context free. This language abstracts the problem of checking that identifiers are declared before their use in a program. That is, the first w in wew represents the declaration of an identifier w . The second w represents its use. While it is beyond the scope of this book to prove it, the non-context-freeness of L_1 directly implies the non-context-freeness of programming languages like Algol and Pascal, which require declaration of identifiers before their use, and which allow identifiers of arbitrary length.

For this reason, a grammar for the syntax of Algol or Pascal does not specify the characters in an identifier. Instead, all identifiers are represented by a token such as `id` in the grammar. In a compiler for such a language, the semantic analysis phase checks that identifiers have been declared before their use. $\square$

Example 4.12. The language $L_2 = \{a^n b^m c^n d^m \mid n \geq 1 \text{ and } m \geq 1\}$ is not context free. That is, L_2 consists of strings in the language generated by the regular expression $a^*b^*c^*d^*$ such that the number of a 's and c 's are equal and the number of b 's and d 's are equal. (Recall a^n means a written n times.) L_2 abstracts the problem of checking that the number of formal parameters in the declaration of a procedure agrees with the number of actual parameters in a use of the procedure. That is, a^n and b^m could represent the formal parameter lists in two procedures declared to have n and m arguments, respectively. Then c^n and d^m represent the actual parameter lists in calls to these two procedures.

Again note that the typical syntax of procedure definitions and uses does not concern itself with counting the number of parameters. For example, the `CALL` statement in a Fortran-like language might be described

$$\begin{aligned} \text{stmt} &\rightarrow \text{call id (expr_list)} \\ \text{expr_list} &\rightarrow \text{expr_list , expr} \\ &\quad | \text{expr} \end{aligned}$$

with suitable productions for *expr*. Checking that the number of actual parameters in the call is correct is usually done during the semantic analysis phase. $\square$

Example 4.13. The language $L_3 = \{a^n b^n c^n \mid n \geq 0\}$, that is, strings in $L(a^* b^* c^*)$ with equal numbers of *a*'s, *b*'s and *c*'s, is not context free. An example of a problem that embeds L_3 is the following. Typeset text uses italics where ordinary typed text uses underlining. In converting a file of text destined to be printed on a line printer to text suitable for a phototypesetter, one has to replace underlined words by italics. An underlined word is a string of letters followed by an equal number of backspaces and an equal number of underscores. If we regard *a* as any letter, *b* as backspace, and *c* as underscore, the language L_3 represents underlined words. The conclusion is that we cannot use a grammar to describe underlined words in this fashion. On the other hand, if we represent an underlined word as a sequence of letter-backspace-underscore triples then we can represent underlined words with the regular expression $(abc)^*$. $\square$

It is interesting to note that languages very similar to L_1 , L_2 , and L_3 are context free. For example, $L'_1 = \{w c w^R \mid w \text{ is in } (a|b)^*\}$, where w^R stands for *w* reversed, is context free. It is generated by the grammar

$$S \rightarrow aSa \mid bSb \mid c$$

The language $L'_2 = \{a^n b^m c^m d^n \mid n \geq 1 \text{ and } m \geq 1\}$ is context free, with grammar

$$\begin{aligned} S &\rightarrow aSd \mid aAd \\ A &\rightarrow bAc \mid bc \end{aligned}$$

Also, $L'_2 = \{a^n b^n c^m d^m \mid n \geq 1 \text{ and } m \geq 1\}$ is context free, with grammar

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aAb \mid ab \\ B &\rightarrow cBd \mid cd \end{aligned}$$

Finally, $L'_3 = \{a^n b^n \mid n \geq 1\}$ is context free, with grammar

$$S \rightarrow aSb \mid ab$$

It is worth noting that L'_3 is the prototypical example of a language not definable by any regular expression. To see this, suppose L'_3 were the language defined by some regular expression. Equivalently, suppose we could construct a DFA *D* accepting L'_3 . *D* must have some finite number of states, say *k*. Consider the sequence of states $s_0, s_1, s_2, \dots, s_k$ entered by *D* having read $\epsilon, a, aa, \dots, a^k$. That is, s_i is the state entered by *D* having read *i* *a*'s.

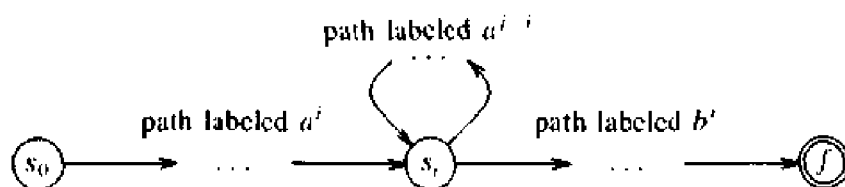


Fig. 4.8. DFA D accepting $a^i b^j$ and $a^j b^i$.

Since D has only k different states, at least two states in the sequence $s_0, s_1, \dots, s_k$ must be the same, say s_i and s_j . From state s_i a sequence of i b 's takes D to an accepting state f , since $a^i b^i$ is in L_3 . But then there is also a path from the initial state s_0 to s_i to f labeled $a^j b^i$, as shown in Fig. 4.8. Thus, D also accepts $a^j b^i$, which is not in L_3 , contradicting the assumption that L_3 is the language accepted by D .

Colloquially, we say that "a finite automaton cannot keep count," meaning that a finite automaton cannot accept a language like L_3 which would require it to keep count of the number of a 's before it sees the b 's. Similarly, we say "a grammar can keep count of two items but not three," since with a grammar we can define L_3 but not L_4 .

4.4 TOP-DOWN PARSING

In this section, we introduce the basic ideas behind top-down parsing and show how to construct an efficient non-backtracking form of top-down parser called a predictive parser. We define the class of LL(1) grammars from which predictive parsers can be constructed automatically. Besides formalizing the discussion of predictive parsers in Section 2.4, we consider nonrecursive predictive parsers. This section concludes with a discussion of error recovery. Bottom-up parsers are discussed in Sections 4.5 – 4.7.

Recursive-Descent Parsing

Top-down parsing can be viewed as an attempt to find a leftmost derivation for an input string. Equivalently, it can be viewed as an attempt to construct a parse tree for the input starting from the root and creating the nodes of the parse tree in preorder. In Section 2.4, we discussed the special case of recursive-descent parsing, called predictive parsing, where no backtracking is required. We now consider a general form of top-down parsing, called recursive descent, that may involve backtracking, that is, making repeated scans of the input. However, backtracking parsers are not seen frequently. One reason is that backtracking is rarely needed to parse programming language constructs. In situations like natural language parsing, backtracking is still not very efficient, and tabular methods such as the dynamic programming algorithm of Exercise 4.63 or the method of Earley [1970] are preferred. See Aho and Ullman [1972b] for a description of general parsing methods.

Backtracking is required in the next example, and we shall suggest a way of keeping track of the input when backtracking takes place.

Example 4.14. Consider the grammar

$$\begin{aligned} S &\rightarrow cAd \\ A &\rightarrow ab \mid a \end{aligned} \quad (4.15)$$

and the input string $w = cad$. To construct a parse tree for this string top-down, we initially create a tree consisting of a single node labeled S . An input pointer points to c , the first symbol of w . We then use the first production for S to expand the tree and obtain the tree of Fig. 4.9(a).

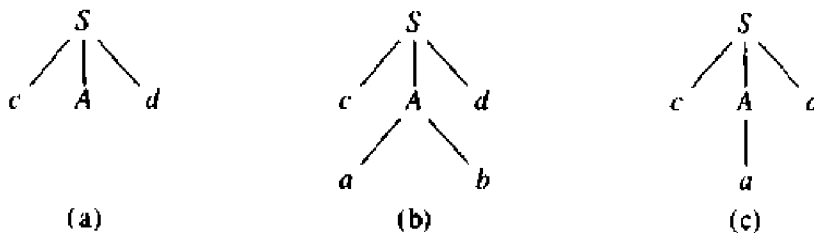


Fig. 4.9. Steps in top-down parse.

The leftmost leaf, labeled c , matches the first symbol of w , so we now advance the input pointer to a , the second symbol of w , and consider the next leaf, labeled A . We can then expand A using the first alternative for A to obtain the tree of Fig. 4.9(b). We now have a match for the second input symbol so we advance the input pointer to d , the third input symbol, and compare d against the next leaf, labeled b . Since b does not match d , we report failure and go back to A to see whether there is another alternative for A that we have not tried but that might produce a match.

In going back to A , we must reset the input pointer to position 2, the position it had when we first came to A , which means that the procedure for A (analogous to the procedure for nonterminals in Fig. 2.17) must store the input pointer in a local variable. We now try the second alternative for A to obtain the tree of Fig. 4.9(c). The leaf a matches the second symbol of w and the leaf d matches the third symbol. Since we have produced a parse tree for w , we halt and announce successful completion of parsing. $\square$

A left-recursive grammar can cause a recursive-descent parser, even one with backtracking, to go into an infinite loop. That is, when we try to expand A , we may eventually find ourselves again trying to expand A without having consumed any input.

Predictive Parsers

In many cases, by carefully writing a grammar, eliminating left recursion from it, and left factoring the resulting grammar, we can obtain a grammar that can

be parsed by a recursive-descent parser that needs no backtracking, i.e., a predictive parser, as discussed in Section 2.4. To construct a predictive parser, we must know, given the current input symbol a and the nonterminal A to be expanded, which one of the alternatives of production $A \rightarrow \alpha_1 | \alpha_2 | \cdots | \alpha_n$ is the unique alternative that derives a string beginning with a . That is, the proper alternative must be detectable by looking at only the first symbol it derives. Flow-of-control constructs in most programming languages, with their distinguishing keywords, are usually detectable in this way. For example, if we have the productions

```

stmt  $\rightarrow$  if expr then stmt else stmt
      | while expr do stmt
      | begin stmt_list end

```

then the keywords **if**, **while**, and **begin** tell us which alternative is the only one that could possibly succeed if we are to find a statement.

Transition Diagrams for Predictive Parsers

In Section 2.4, we discussed the implementation of predictive parsers by recursive procedures, e.g., those of Fig. 2.17. Just as a transition diagram was seen in Section 3.4 to be a useful plan or flowchart for a lexical analyzer, we can create a transition diagram as a plan for a predictive parser.

Several differences between the transition diagrams for a lexical analyzer and a predictive parser are immediately apparent. In the case of the parser, there is one diagram for each nonterminal. The labels of edges are tokens and nonterminals. A transition on a token (terminal) means we should take that transition if that token is the next input symbol. A transition on a nonterminal A is a call of the procedure for A .

To construct the transition diagram of a predictive parser from a grammar, first eliminate left recursion from the grammar, and then left factor the grammar. Then for each nonterminal A do the following:

1. Create an initial and final (return) state.
2. For each production $A \rightarrow X_1 X_2 \cdots X_n$, create a path from the initial to the final state, with edges labeled $X_1, X_2, \dots, X_n$.

The predictive parser working off the transition diagrams behaves as follows. It begins in the start state for the start symbol. If after some actions it is in state s with an edge labeled by terminal a to state t , and if the next input symbol is a , then the parser moves the input cursor one position right and goes to state t . If, on the other hand, the edge is labeled by a nonterminal A , the parser instead goes to the start state for A , without moving the input cursor. If it ever reaches the final state for A , it immediately goes to state t , in effect having "read" A from the input during the time it moved from state s to t . Finally, if there is an edge from s to t labeled ϵ , then from state s the parser immediately goes to state t , without advancing the input.

A predictive parsing program based on a transition diagram attempts to match terminal symbols against the input, and makes a potentially recursive procedure call whenever it has to follow an edge labeled by a nonterminal. A nonrecursive implementation can be obtained by stacking the states s when there is a transition on a nonterminal out of s , and popping the stack when the final state for a nonterminal is reached. We shall discuss the implementation of transition diagrams in more detail shortly.

The above approach works if the given transition diagram does not have nondeterminism, in the sense that there is more than one transition from a state on the same input. If ambiguity occurs, we may be able to resolve it in an ad-hoc way, as in the next example. If the nondeterminism cannot be eliminated, we cannot build a predictive parser, but we could build a recursive-descent parser using backtracking to systematically try all possibilities, if that were the best parsing strategy we could find.

Example 4.15. Figure 4.10 contains a collection of transition diagrams for grammar (4.11). The only ambiguities concern whether or not to take an ϵ -edge. If we interpret the edges out of the initial state for E' as saying take the transition on $+$ whenever that is the next input and take the transition on ϵ otherwise, and make the analogous assumption for T' , then the ambiguity is removed, and we can write a predictive parsing program for grammar (4.11). $\square$

Transition diagrams can be simplified by substituting diagrams in one another; these substitutions are similar to the transformations on grammars used in Section 2.5. For example, in Fig. 4.11(a), the call of E' on itself has been replaced by a jump to the beginning of the diagram for E' .

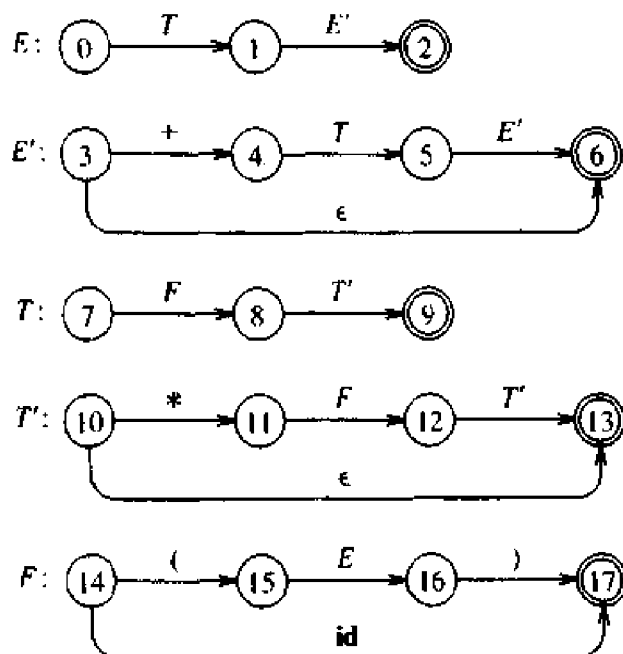


Fig. 4.10. Transition diagrams for grammar (4.11).

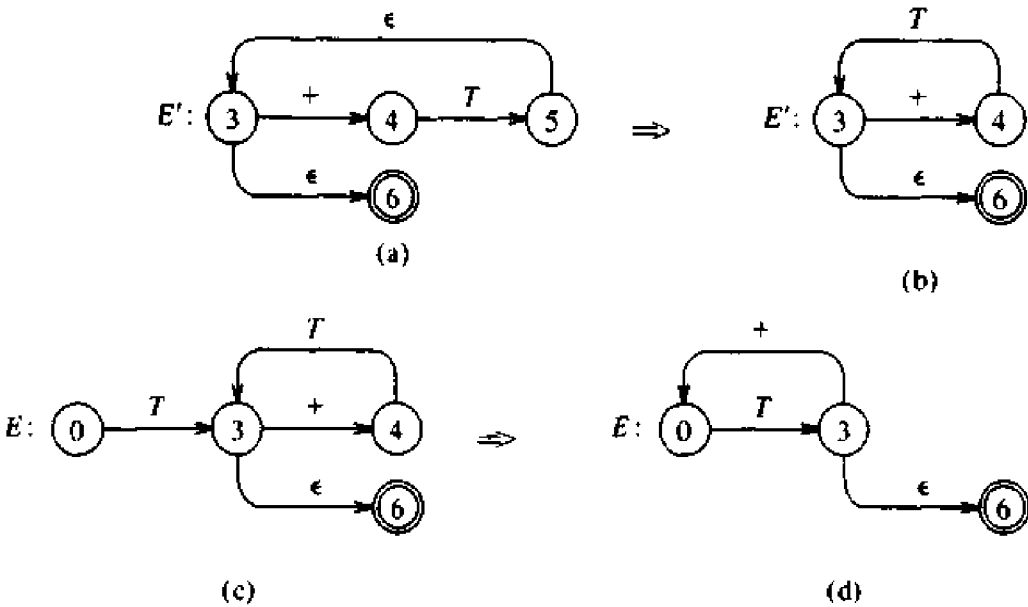


Fig. 4.11. Simplified transition diagrams.

Figure 4.11(b) shows an equivalent transition diagram for E' . We may then substitute the diagram of Fig. 4.11(b) for the transition on E' in the diagram for E in Fig. 4.10, yielding the diagram of Fig. 4.11(c). Lastly, we observe that the first and third nodes in Fig. 4.11(c) are equivalent and we merge them. The result, Fig. 4.11(d), is repeated as the first diagram in Fig. 4.12. The same techniques apply to the diagrams for T and T' . The complete set of resulting diagrams is shown in Fig. 4.12. A C implementation of this predictive parser runs 20-25% faster than a C implementation of Fig. 4.10.

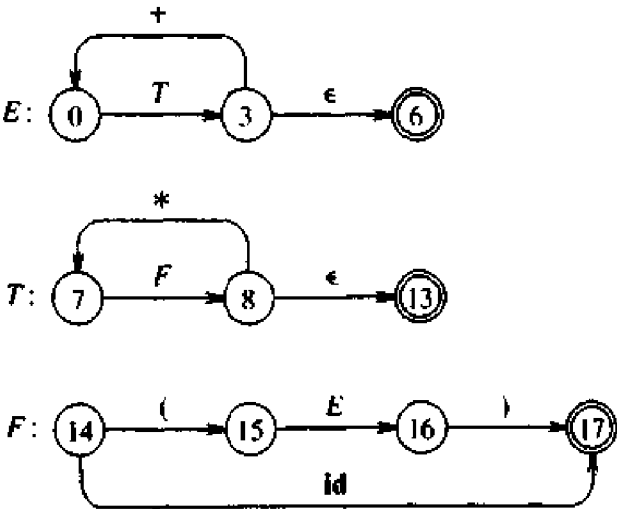


Fig. 4.12. Simplified transition diagrams for arithmetic expressions.

Nonrecursive Predictive Parsing

It is possible to build a nonrecursive predictive parser by maintaining a stack explicitly, rather than implicitly via recursive calls. The key problem during predictive parsing is that of determining the production to be applied for a nonterminal. The nonrecursive parser in Fig. 4.13 looks up the production to be applied in a parsing table. In what follows, we shall see how the table can be constructed directly from certain grammars.

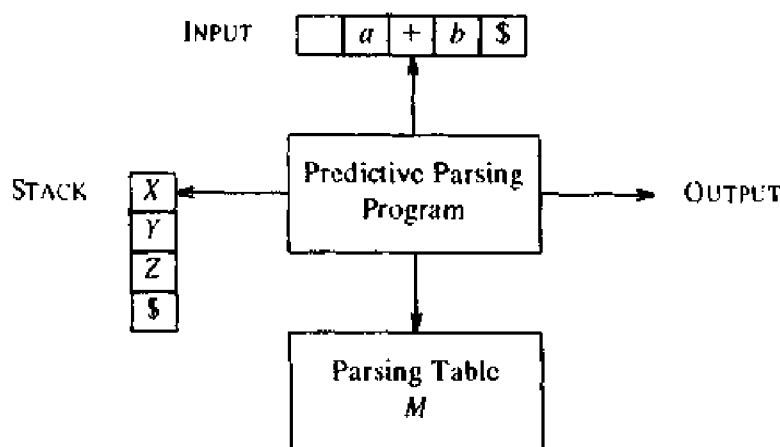


Fig. 4.13. Model of a nonrecursive predictive parser.

A table-driven predictive parser has an input buffer, a stack, a parsing table, and an output stream. The input buffer contains the string to be parsed, followed by \$, a symbol used as a right endmarker to indicate the end of the input string. The stack contains a sequence of grammar symbols with \$ on the bottom, indicating the bottom of the stack. Initially, the stack contains the start symbol of the grammar on top of \$. The parsing table is a two-dimensional array $M[A, a]$, where A is a nonterminal, and a is a terminal or the symbol \$.

The parser is controlled by a program that behaves as follows. The program considers X , the symbol on top of the stack, and a , the current input symbol. These two symbols determine the action of the parser. There are three possibilities.

1. If $X = a = \$$, the parser halts and announces successful completion of parsing.
2. If $X = a \neq \$$, the parser pops X off the stack and advances the input pointer to the next input symbol.
3. If X is a nonterminal, the program consults entry $M[X, a]$ of the parsing table M . This entry will be either an X -production of the grammar or an error entry. If, for example, $M[X, a] = \{X \rightarrow UVW\}$, the parser replaces X on top of the stack by WVU (with U on top). As output, we shall

assume that the parser just prints the production used; any other code could be executed here. If $M[X, a] = \text{error}$, the parser calls an error recovery routine.

The behavior of the parser can be described in terms of its *configurations*, which give the stack contents and the remaining input.

Algorithm 4.3. Nonrecursive predictive parsing.

Input. A string w and a parsing table M for grammar G .

Output. If w is in $L(G)$, a leftmost derivation of w ; otherwise, an error indication.

Method. Initially, the parser is in a configuration in which it has $\$S$ on the stack with S , the start symbol of G on top, and $w\$$ in the input buffer. The program that utilizes the predictive parsing table M to produce a parse for the input is shown in Fig. 4.14. $\square$

```

set  $ip$  to point to the first symbol of  $w\$$ ;
repeat
    let  $X$  be the top stack symbol and  $a$  the symbol pointed to by  $ip$ ;
    if  $X$  is a terminal or  $\$$  then
        if  $X = a$  then
            pop  $X$  from the stack and advance  $ip$ 
        else error()
    else /*  $X$  is a nonterminal */
        if  $M[X, a] = X \rightarrow Y_1 Y_2 \cdots Y_k$  then begin
            pop  $X$  from the stack;
            push  $Y_k, Y_{k-1}, \dots, Y_1$  onto the stack, with  $Y_1$  on top;
            output the production  $X \rightarrow Y_1 Y_2 \cdots Y_k$ 
        end
        else error()
until  $X = \$$  /* stack is empty */

```

Fig. 4.14. Predictive parsing program.

Example 4.16. Consider the grammar (4.11) from Example 4.8. A predictive parsing table for this grammar is shown in Fig. 4.15. Blanks are error entries; non-blanks indicate a production with which to expand the top nonterminal on the stack. Note that we have not yet indicated how these entries could be selected, but we shall do so shortly.

With input $\text{id} + \text{id} * \text{id}$ the predictive parser makes the sequence of moves in Fig. 4.16. The input pointer points to the leftmost symbol of the string in the INPUT column. If we observe the actions of this parser carefully, we see that it is tracing out a leftmost derivation for the input, that is, the productions output are those of a leftmost derivation. The input symbols that have

NONTER-MINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
<i>E</i>	<i>E</i> → <i>TE'</i>			<i>E</i> → <i>TE'</i>		
<i>E'</i>		<i>E'</i> → + <i>TE'</i>			<i>E'</i> → ε	<i>E'</i> → ε
<i>T</i>	<i>T</i> → <i>FT'</i>			<i>T</i> → <i>FT'</i>		
<i>T'</i>		<i>T'</i> → ε	<i>T'</i> → * <i>FT'</i>		<i>T'</i> → ε	<i>T'</i> → ε
<i>F</i>	<i>F</i> → <i>id</i>			<i>F</i> → (<i>E</i>)		

Fig. 4.15. Parsing table *M* for grammar (4.11).

already been scanned, followed by the grammar symbols on the stack (from the top to bottom), make up the left-sentential forms in the derivation. □

STACK	INPUT	OUTPUT
<i>\$E</i>	<i>id + id * id\$</i>	
<i>\$E'T</i>	<i>id + id * id\$</i>	<i>E → TE'</i>
<i>\$E'T'F</i>	<i>id + id * id\$</i>	<i>T → FT'</i>
<i>\$E'T'id</i>	<i>id + id * id\$</i>	<i>F → id</i>
<i>\$E'T'</i>	<i>+ id * id\$</i>	
<i>\$E'</i>	<i>+ id * id\$</i>	<i>T' → ε</i>
<i>\$E'T +</i>	<i>+ id * id\$</i>	<i>E' → +TE'</i>
<i>\$E'T</i>	<i>id * id\$</i>	
<i>\$E'T'F</i>	<i>id * id\$</i>	<i>T → FT'</i>
<i>\$E'T'id</i>	<i>id * id\$</i>	<i>F → id</i>
<i>\$E'T'</i>	<i>* id\$</i>	
<i>\$E'T'F*</i>	<i>* id\$</i>	<i>T' → *FT'</i>
<i>\$E'T'F</i>	<i>id\$</i>	
<i>\$E'T'id</i>	<i>id\$</i>	<i>F → id</i>
<i>\$E'T'</i>	<i>\$</i>	
<i>\$E'</i>	<i>\$</i>	<i>T' → ε</i>
<i>\$</i>	<i>\$</i>	<i>E' → ε</i>

Fig. 4.16. Moves made by predictive parser on input *id + id * id*.

FIRST and FOLLOW

The construction of a predictive parser is aided by two functions associated with a grammar *G*. These functions, FIRST and FOLLOW, allow us to fill in the entries of a predictive parsing table for *G*, whenever possible. Sets of tokens yielded by the FOLLOW function can also be used as synchronizing tokens during panic-mode error recovery.

If α is any string of grammar symbols, let FIRST(α) be the set of terminals

that begin the strings derived from α . If $\alpha \xRightarrow{*} \epsilon$, then ϵ is also in $\text{FIRST}(\alpha)$.

Define $\text{FOLLOW}(A)$, for nonterminal A , to be the set of terminals a that can appear immediately to the right of A in some sentential form, that is, the set of terminals a such that there exists a derivation of the form $S \xRightarrow{*} \alpha A a \beta$ for some α and β . Note that there may, at some time during the derivation, have been symbols between A and a , but if so, they derived ϵ and disappeared. If A can be the rightmost symbol in some sentential form, then $\$$ is in $\text{FOLLOW}(A)$.

To compute $\text{FIRST}(X)$ for all grammar symbols X , apply the following rules until no more terminals or ϵ can be added to any FIRST set.

1. If X is terminal, then $\text{FIRST}(X)$ is $\{X\}$.
2. If $X \rightarrow \epsilon$ is a production, then add ϵ to $\text{FIRST}(X)$.
3. If X is nonterminal and $X \rightarrow Y_1 Y_2 \cdots Y_k$ is a production, then place a in $\text{FIRST}(X)$ if for some i , a is in $\text{FIRST}(Y_i)$, and ϵ is in all of $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$; that is, $Y_1 \cdots Y_{i-1} \xRightarrow{*} \epsilon$. If ϵ is in $\text{FIRST}(Y_j)$ for all $j = 1, 2, \dots, k$, then add ϵ to $\text{FIRST}(X)$. For example, everything in $\text{FIRST}(Y_1)$ is surely in $\text{FIRST}(X)$. If Y_1 does not derive ϵ , then we add nothing more to $\text{FIRST}(X)$, but if $Y_1 \xRightarrow{*} \epsilon$, then we add $\text{FIRST}(Y_2)$ and so on.

Now, we can compute FIRST for any string $X_1 X_2 \cdots X_n$ as follows. Add to $\text{FIRST}(X_1 X_2 \cdots X_n)$ all the non- ϵ symbols of $\text{FIRST}(X_1)$. Also add the non- ϵ symbols of $\text{FIRST}(X_2)$ if ϵ is in $\text{FIRST}(X_1)$, the non- ϵ symbols of $\text{FIRST}(X_3)$ if ϵ is in both $\text{FIRST}(X_1)$ and $\text{FIRST}(X_2)$, and so on. Finally, add ϵ to $\text{FIRST}(X_1 X_2 \cdots X_n)$ if, for all i , $\text{FIRST}(X_i)$ contains ϵ .

To compute $\text{FOLLOW}(A)$ for all nonterminals A , apply the following rules until nothing can be added to any FOLLOW set.

1. Place $\$$ in $\text{FOLLOW}(S)$, where S is the start symbol and $\$$ is the input right endmarker.
2. If there is a production $A \rightarrow \alpha B \beta$, then everything in $\text{FIRST}(\beta)$ except for ϵ is placed in $\text{FOLLOW}(B)$.
3. If there is a production $A \rightarrow \alpha B$, or a production $A \rightarrow \alpha B \beta$ where $\text{FIRST}(\beta)$ contains ϵ (i.e., $\beta \xRightarrow{*} \epsilon$), then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

Example 4.17. Consider again grammar (4.11), repeated below:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Then:

$$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}.$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

$$\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{), \$ \}$$

$$\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{ +,), \$ \}$$

$$\text{FOLLOW}(F) = \{ +, *,), \$ \}$$

For example, **id** and left parenthesis are added to $\text{FIRST}(F)$ by rule (3) in the definition of FIRST with $i = 1$ in each case, since $\text{FIRST}(\text{id}) = \{\text{id}\}$ and $\text{FIRST}('(') = \{ (\}$ by rule (1). Then by rule (3) with $i = 1$, the production $T \rightarrow FT'$ implies that **id** and left parenthesis are in $\text{FIRST}(T)$ as well. As another example, ϵ is in $\text{FIRST}(E')$ by rule (2).

To compute FOLLOW sets, we put $\$$ in $\text{FOLLOW}(E)$ by rule (1) for FOLLOW . By rule (2) applied to production $F \rightarrow (E)$, the right parenthesis is also in $\text{FOLLOW}(E)$. By rule (3) applied to production $E \rightarrow TE'$, $\$$ and right parenthesis are in $\text{FOLLOW}(E')$. Since $E' \xRightarrow{*} \epsilon$, they are also in $\text{FOLLOW}(T)$. For a last example of how the FOLLOW rules are applied, the production $E \rightarrow TE'$ implies, by rule (2), that everything other than ϵ in $\text{FIRST}(E')$ must be placed in $\text{FOLLOW}(T)$. We have already seen that $\$$ is in $\text{FOLLOW}(T)$. $\square$

Construction of Predictive Parsing Tables

The following algorithm can be used to construct a predictive parsing table for a grammar G . The idea behind the algorithm is the following. Suppose $A \rightarrow \alpha$ is a production with a in $\text{FIRST}(\alpha)$. Then, the parser will expand A by α when the current input symbol is a . The only complication occurs when $\alpha = \epsilon$ or $\alpha \xRightarrow{*} \epsilon$. In this case, we should again expand A by α if the current input symbol is in $\text{FOLLOW}(A)$, or if the $\$$ on the input has been reached and $\$$ is in $\text{FOLLOW}(A)$.

Algorithm 4.4. Construction of a predictive parsing table.

Input. Grammar G .

Output. Parsing table M .

Method.

1. For each production $A \rightarrow \alpha$ of the grammar, do steps 2 and 3.
2. For each terminal a in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$.
3. If ϵ is in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, b]$ for each terminal b in $\text{FOLLOW}(A)$. If ϵ is in $\text{FIRST}(\alpha)$ and $\$$ is in $\text{FOLLOW}(A)$, add $A \rightarrow \alpha$ to $M[A, \$]$.
4. Make each undefined entry of M be **error**.

Example 4.18. Let us apply Algorithm 4.4 to grammar (4.11). Since $\text{FIRST}(TE') = \text{FIRST}(T) = \{ (, \text{id} \}$, production $E \rightarrow TE'$ causes $M[E, (]$ and $M[E, \text{id}]$ to acquire the entry $E \rightarrow TE'$.

Production $E' \rightarrow +TE'$ causes $M[E', +]$ to acquire $E' \rightarrow +TE'$. Production $E' \rightarrow \epsilon$ causes $M[E',)]$ and $M[E', \$]$ to acquire $E' \rightarrow \epsilon$ since $\text{FOLLOW}(E') = \{), \$ \}$.

The parsing table produced by Algorithm 4.4 for grammar (4.11) was shown in Fig. 4.15. $\square$

LL(1) Grammars

Algorithm 4.4 can be applied to any grammar G to produce a parsing table M . For some grammars, however, M may have some entries that are multiply-defined. For example, if G is left recursive or ambiguous, then M will have at least one multiply-defined entry.

Example 4.19. Let us consider grammar (4.13) from Example 4.10 again; it is repeated here for convenience.

$$\begin{aligned} S &\rightarrow iEtSS' \mid a \\ S' &\rightarrow eS \mid \epsilon \\ E &\rightarrow b \end{aligned}$$

The parsing table for this grammar is shown in Fig. 4.17.

NONTERMINAL	INPUT SYMBOL					
	a	b	e	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

Fig. 4.17. Parsing table M for grammar (4.13).

The entry for $M[S', e]$ contains both $S' \rightarrow eS$ and $S' \rightarrow \epsilon$, since $\text{FOLLOW}(S') = \{ e, \$ \}$. The grammar is ambiguous and the ambiguity is manifested by a choice in what production to use when an e (else) is seen. We can resolve the ambiguity if we choose $S' \rightarrow eS$. This choice corresponds to associating else's with the closest previous then's. Note that the choice $S' \rightarrow \epsilon$ would prevent e from ever being put on the stack or removed from the input, and is therefore surely wrong. $\square$

A grammar whose parsing table has no multiply-defined entries is said to be LL(1). The first "L" in LL(1) stands for scanning the input from left to right, the second "L" for producing a leftmost derivation, and the "1" for using one input symbol of lookahead at each step to make parsing action

decisions. It can be shown that Algorithm 4.4 produces for every LL(1) grammar G a parsing table that parses all and only the sentences of G .

LL(1) grammars have several distinctive properties. No ambiguous or left-recursive grammar can be LL(1). It can also be shown that a grammar G is LL(1) if and only if whenever $A \rightarrow \alpha \mid \beta$ are two distinct productions of G the following conditions hold:

1. For no terminal a do both α and β derive strings beginning with a .
2. At most one of α and β can derive the empty string.
3. If $\beta \xRightarrow{*} \epsilon$, then α does not derive any string beginning with a terminal in FOLLOW(A).

Clearly, grammar (4.11) for arithmetic expressions is LL(1). Grammar (4.13), modeling if-then-else statements, is not.

There remains the question of what should be done when a parsing table has multiply-defined entries. One recourse is to transform the grammar by eliminating all left recursion and then left factoring whenever possible, hoping to produce a grammar for which the parsing table has no multiply-defined entries. Unfortunately, there are some grammars for which no amount of alteration will yield an LL(1) grammar. Grammar (4.13) is one such example; its language has no LL(1) grammar at all. As we saw, we can still parse (4.13) with a predictive parser by arbitrarily making $M[S', e] = \{S' \rightarrow eS\}$. In general, there are no universal rules by which multiply-defined entries can be made single-valued without affecting the language recognized by the parser.

The main difficulty in using predictive parsing is in writing a grammar for the source language such that a predictive parser can be constructed from the grammar. Although left-recursion elimination and left factoring are easy to do, they make the resulting grammar hard to read and difficult to use for translation purposes. To alleviate some of this difficulty, a common organization for a parser in a compiler is to use a predictive parser for control constructs and to use operator precedence (discussed in Section 4.6) for expressions. However, if an LR parser generator, as discussed in Section 4.9, is available, one can get all the benefits of predictive parsing and operator precedence automatically.

Error Recovery in Predictive Parsing

The stack of a nonrecursive predictive parser makes explicit the terminals and nonterminals that the parser hopes to match with the remainder of the input. We shall therefore refer to symbols on the parser stack in the following discussion. An error is detected during predictive parsing when the terminal on top of the stack does not match the next input symbol or when nonterminal A is on top of the stack, a is the next input symbol, and the parsing table entry $M[A, a]$ is empty.

Panic-mode error recovery is based on the idea of skipping symbols on the the input until a token in a selected set of synchronizing tokens appears. Its